

# Living Map Installation and Operation Guide

From an empty computer to a complete mission:  
every command, in order

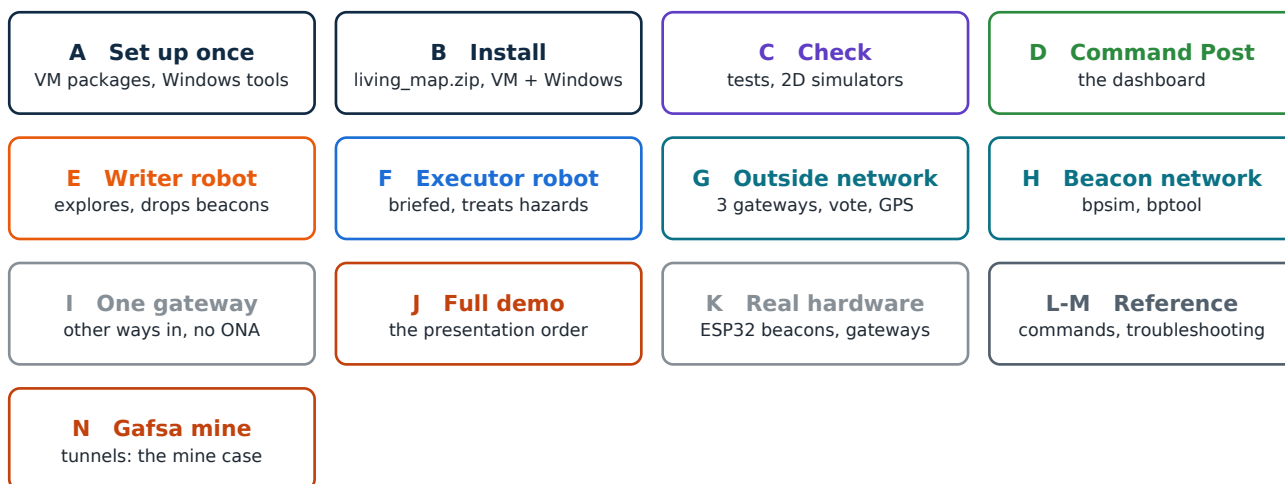
# Contents

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>How to use this guide</b>                                    | <b>4</b>  |
| What you deliver and install: living_map.zip                    | 4         |
| Where each command runs                                         | 5         |
| Rules that avoid most problems                                  | 5         |
| <b>A. One-time setup</b>                                        | <b>6</b>  |
| A.1 Ubuntu VM                                                   | 6         |
| A.2 Windows                                                     | 6         |
| A.3 Make the VM lighter and easier (optional)                   | 6         |
| <b>B. Install the project</b>                                   | <b>7</b>  |
| B.1 In the VM                                                   | 7         |
| B.2 On Windows                                                  | 7         |
| B.3 Updating later                                              | 8         |
| <b>C. Check that everything works (no Gazebo)</b>               | <b>8</b>  |
| C.1 On Windows (double-click)                                   | 8         |
| C.2 The same in the VM                                          | 8         |
| C.3 Options of the 2D simulators                                | 9         |
| <b>D. The Command Post dashboard</b>                            | <b>10</b> |
| D.1 Start it                                                    | 10        |
| D.2 Where the dashboard is, from each place                     | 11        |
| D.3 Using it                                                    | 11        |
| D.4 Check the server from a terminal                            | 11        |
| <b>E. The Writer robot</b>                                      | <b>12</b> |
| E.1 Start it                                                    | 12        |
| E.2 Watch what it does (new terminals)                          | 12        |
| E.3 Stop it and keep the mission                                | 13        |
| E.4 Change a parameter                                          | 14        |
| E.5 Start the Writer by hand, one terminal per piece (advanced) | 14        |
| E.6 Drive by hand and test the trapdoor (optional)              | 15        |
| <b>F. The Executor robot</b>                                    | <b>15</b> |
| F.1 Start it                                                    | 15        |
| F.2 Watch it (new terminals)                                    | 16        |
| F.3 Brief it from the Command Post (option ona)                 | 16        |
| F.4 Change what it does                                         | 17        |
| F.5 Beacon-graph navigation (Dijkstra / A*): ready for later    | 18        |
| F.6 Start the Executor by hand (advanced)                       | 18        |
| <b>G. The Outside Network Area (three gateways)</b>             | <b>19</b> |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| G.1 The zone simulator: a whole mission, three gateways . . . . .          | 19        |
| G.2 The beacon network simulator through the vote . . . . .                | 19        |
| G.3 With the Gazebo robots . . . . .                                       | 20        |
| G.4 Look inside . . . . .                                                  | 20        |
| <b>H. The beacon network (LMB2) . . . . .</b>                              | <b>21</b> |
| H.1 Run the network simulator . . . . .                                    | 21        |
| H.2 Tests and stress matrix . . . . .                                      | 22        |
| H.3 Work with single frames (bptool) . . . . .                             | 22        |
| <b>I. One gateway, no ONA: other ways into the dashboard . . . . .</b>     | <b>23</b> |
| I.1 Simulated network to dashboard . . . . .                               | 23        |
| I.2 The Writer (Gazebo) to dashboard, LMB2 records . . . . .               | 23        |
| I.3 The Writer to dashboard, original contract (ros_bridge.py) . . . . .   | 24        |
| I.4 A real gateway board to dashboard . . . . .                            | 24        |
| <b>J. The full demo, in order . . . . .</b>                                | <b>24</b> |
| J.1 Make it fast enough for a demo . . . . .                               | 24        |
| J.2 The order of the presentation . . . . .                                | 25        |
| <b>K. Real hardware: beacons and gateways . . . . .</b>                    | <b>26</b> |
| K.1 Arduino IDE 2 (Windows), once . . . . .                                | 26        |
| K.2 Flash the boards . . . . .                                             | 26        |
| K.3 Talk to a board (Serial Monitor, 115200 baud) . . . . .                | 26        |
| K.4 Provision beacons from the Writer . . . . .                            | 27        |
| K.5 Three real gateways into the ONA . . . . .                             | 27        |
| <b>L. Command reference . . . . .</b>                                      | <b>28</b> |
| L.1 The commands you will use most . . . . .                               | 28        |
| L.2 ROS 2 and Gazebo tools . . . . .                                       | 28        |
| <b>M. Troubleshooting . . . . .</b>                                        | <b>29</b> |
| M.1 The robot does not drive, or the map in RViz turns or smears . . . . . | 29        |
| M.2 A hazard block is not detected . . . . .                               | 30        |
| M.3 The ONA and the dashboard . . . . .                                    | 30        |
| M.4 Other problems . . . . .                                               | 30        |
| M.5 Logs and files . . . . .                                               | 31        |
| <b>N. The Gafsa mine (tunnels) . . . . .</b>                               | <b>32</b> |
| N.1 Without ROS . . . . .                                                  | 32        |
| N.2 With the Gazebo robots . . . . .                                       | 33        |
| N.3 Change the mine . . . . .                                              | 33        |
| N.4 Troubleshooting in the mine . . . . .                                  | 33        |

# How to use this guide

This guide takes you from an empty computer to a complete Living Map mission: the **Writer** robot explores a contaminated building and drops LoRa beacons, the beacon network passes the records around, **three gateways** and the **Outside Network Area (ONA)** check them, locate the robots and forward everything to the **Command Post** dashboard, and the **Executor** robot, briefed from the dashboard, follows the beacons to treat the hazards. Every command you can use is here, in the order you need it, with what it does and where to type it. The companion document *2\_Living\_Map\_Architecture.pdf* explains how each piece works inside.



A and B once, in this order. C to I and N in any order. J is the order for a presentation.  
The parts of this guide.

## What you deliver and install: living\_map.zip

One zip holds the whole project. Unzip it in the Ubuntu VM (`~/living_map`) and on Windows (`C:\tsyp\living_map`): the same folder works in both places.

| in living_map/                     | what it is                                                                                                                                                                                                                              |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| START_HERE.md                      | the first page to read: the folder, the first commands                                                                                                                                                                                  |
| COMMANDS.txt                       | every command of this guide, one per line, same sections: copy commands from there                                                                                                                                                      |
| docs/1_Living_Map_Guide.pdf        | this guide: install and run everything                                                                                                                                                                                                  |
| docs/2_Living_Map_Architecture.pdf | how everything works                                                                                                                                                                                                                    |
| docs/3_Gafsa_Mine_Case.pdf         | the same system in a mine (Gafsa): dangers, sensors, robots, radio, dashboard                                                                                                                                                           |
| writer_robot/                      | the ROS 2 package of both robots (Writer and Executor), Gazebo worlds, 2D simulators and tests, <code>install.sh</code> , the run scripts, Windows <code>.bat</code> launchers                                                          |
| beacon_net/                        | LMB2: the beacon protocol in C, the ESP32 firmware (beacon and gateway), the network simulator <code>bpsim</code> , <code>bptool</code> , the Python mirror and the one-gateway bridge, Windows <code>.exe</code> and <code>.bat</code> |
| ona/                               | the Outside Network Area: three gateways, the vote, the robots located, frame translation, LTE / satellite queue, briefings; the zone simulator and its tests; the GALERIA originals                                                    |
| command_post/                      | the Command Post dashboard (Node.js) and its scripted demo                                                                                                                                                                              |

## Where each command runs

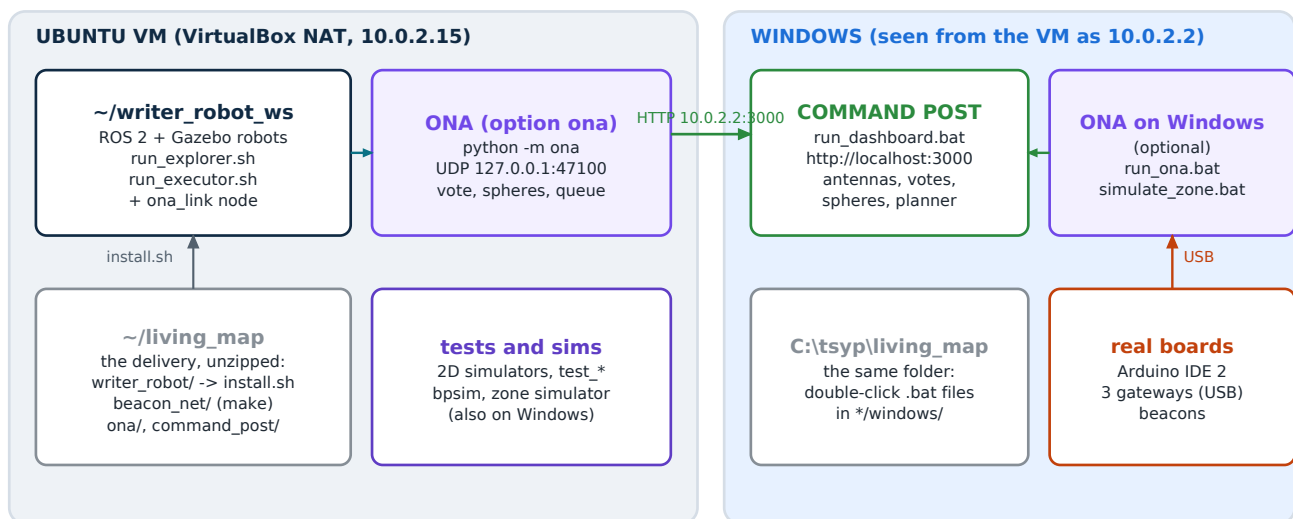
Every command has a coloured tag that tells you where to type it:

| tag            | meaning                                                                                                                          |
|----------------|----------------------------------------------------------------------------------------------------------------------------------|
| VM terminal    | a terminal in the Ubuntu VM (Ctrl+Alt+T opens one). ROS is loaded by your <code>~/ .bashrc</code> (A.1, B.1) and by the scripts. |
| Windows        | a double-click on a <code>.bat</code> file, or a line in a Command Prompt ( <code>cmd</code> ) window                            |
| Browser        | a web browser (on Windows unless it says otherwise)                                                                              |
| Serial / board | the Serial Monitor of the Arduino IDE (115200 baud), talking to an ESP32 board                                                   |
| VM or Windows  | works the same in both (on Windows type <code>py -3</code> instead of <code>python3</code> )                                     |

## Rules that avoid most problems

- **Every command is one line.** Paste one line at a time and press Enter. Long commands wrap over two or three lines on these pages but are still one line: copy them from `COMMANDS.txt` (open it in gedit in the VM or in Notepad on Windows), because copying from a PDF can cut them into pieces.
- `~` is your home folder in the VM (`/home/<you>`). Paths starting with `C:\` are on Windows.
- Ctrl-C stops the program running in a terminal. The robot scripts stop everything they started when you press Ctrl-C.
- Anything that keeps running (a robot, the dashboard, a log you follow) needs its own terminal or window. The guide says "new terminal" when you need one.
- When something looks wrong, look at the log first. Part M lists what each log tells you.

Copy `living_map.zip` into the VM (`~/Downloads` or a shared folder) and unzip it on Windows too.



Where everything runs: the robots in the VM, everything else in the VM or on Windows. The VM reaches Windows at 10.0.2.2.

# A. One-time setup

## A.1 Ubuntu VM

ROS 2 Lyrical and Gazebo Jetty are already installed in the VM. These lines add everything the Living Map needs on top; running them again is harmless.

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>sudo apt update</code><br>refresh the package lists                                                                                                                                                                                                                                                                                                                                                                          |
| <b>VM terminal</b> | <code>sudo apt install -y ros-lyrical-ros-gz ros-lyrical-robot-localization<br/>ros-lyrical-slam-toolbox ros-lyrical-xacro ros-lyrical-robot-state-publisher<br/>ros-lyrical-joint-state-publisher ros-lyrical-cv-bridge python3-opencv<br/>ros-lyrical-teleop-twist-keyboard ros-lyrical-tf2-tools</code><br>the ROS packages the robots use: Gazebo bridge, EKF, SLAM, URDF tools, camera conversion, keyboard driving, TF tools |
| <b>VM terminal</b> | <code>sudo apt install -y build-essential unzip curl nodejs npm python3-numpy<br/>python3-serial python3-matplotlib</code><br>C compiler (beacon simulator), unzip, curl, Node.js (dashboard), numpy (ONA, simulators), serial ports, plots                                                                                                                                                                                        |
| <b>VM terminal</b> | <code>grep -q 'opt/ros/lyrical' ~/.bashrc    echo 'source /opt/ros/lyrical/setup.bash' &gt;&gt;<br/>~/.bashrc</code><br>every new terminal loads ROS 2 (adds the line only if it is missing)                                                                                                                                                                                                                                       |
| <b>VM terminal</b> | <code>sudo usermod -aG dialout,vboxsf \$USER</code><br>lets you use USB serial ports and VirtualBox shared folders; takes effect after a reboot                                                                                                                                                                                                                                                                                    |

## A.2 Windows

| install                                        | why                                       | where from            |
|------------------------------------------------|-------------------------------------------|-----------------------|
| Node.js LTS                                    | the Command Post dashboard                | nodejs.org            |
| Python 3 (tick <b>Add python.exe to PATH</b> ) | ONA, gateway bridge, 2D simulators, tests | python.org/downloads  |
| Arduino IDE 2 + USB driver (CP210x or CH340)   | only for real ESP32 boards (part K)       | arduino.cc            |
| VS Code (optional)                             | reading and editing the code              | code.visualstudio.com |

The Python packages are installed by two .bat files once the folder is unzipped (B.2).

## A.3 Make the VM lighter and easier (optional)

- VM powered off → Settings → System: about half your CPU cores (4 or more) and 6–8 GB RAM. Gazebo is CPU-bound.
- Share a Windows folder instead of copying through Downloads: Devices → Shared Folders, add C:\tsyp, tick Auto-mount. After the usermod line above and a reboot it appears at /media/sf\_tsyp. Keep the ROS workspace itself inside the VM (builds on a shared folder are slow).
- Run the robots with headless (no Gazebo window): the single biggest speed-up. RViz still shows everything.

## B. Install the project

### B.1 In the VM

Copy `living_map.zip` into `~/Downloads` in the VM (drag and drop, browser download, or from the shared folder with the first line below), then run the lines in order.

|             |                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VM terminal | <code>cp /media/sf_tsyp/living_map.zip ~/Downloads/</code><br>only if you use the shared folder of A.3                                                                                                                                                                                                                                                                                                           |
| VM terminal | <code>cd ~ &amp;&amp; rm -rf ~/living_map &amp;&amp; unzip -o ~/Downloads/living_map.zip</code><br>creates <code>~/living_map/</code> with <code>writer_robot/</code> , <code>beacon_net/</code> , <code>ona/</code> , <code>command_post/</code> , <code>docs/</code> . A previous <code>~/living_map</code> is replaced (keep your own files elsewhere).                                                       |
| VM terminal | <code>bash ~/living_map/writer_robot/install.sh</code><br>copies the robots package (and a copy of <code>ona/</code> ) to <code>~/writer_robot_ws/src/writer_robot</code> , puts the run scripts in <code>~/writer_robot_ws</code> , and builds. It must end with <b>Summary: 1 package finished</b> and the "Installed" banner. A package already there is moved to <code>~/writer_robot_backups/</code> first. |
| VM terminal | <code>grep -q 'writer_robot_ws/install' ~/.bashrc    echo 'source</code><br><code>~/writer_robot_ws/install/setup.bash' &gt;&gt; ~/.bashrc</code><br>every new terminal also loads the robots package (only if the line is missing)                                                                                                                                                                              |
| VM terminal | <code>make -C ~/living_map/beacon_net/host</code><br>builds the beacon network simulator <code>bpsim</code> and the frame tool <code>bptool</code>                                                                                                                                                                                                                                                               |
| VM terminal | <code>cd ~/living_map/command_post &amp;&amp; npm install</code><br>only if you run the dashboard in the VM: downloads its Node packages (once, needs internet)                                                                                                                                                                                                                                                  |
| VM terminal | <code>python3 -c "import numpy, serial; print('numpy', numpy.__version__, 'and pyserial</code><br><code>OK')"</code><br>the ONA needs <code>numpy</code> ; <code>pyserial</code> only for real boards. If it fails: the second <code>apt install</code> line of A.1.                                                                                                                                             |

Then **close the terminal and open a new one** (it loads the package). What the installer puts in `~/writer_robot_ws`:

| file or folder                 | use                                                                                                       |
|--------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>run_explorer.sh</code>   | the Writer: explores, drops beacons, records the mission (part E)                                         |
| <code>run_executor.sh</code>   | the Executor: follows the beacons, briefed from the Command Post (part F)                                 |
| <code>stop_demo.sh</code>      | kills every simulation process (Gazebo, SLAM, robots, ONA, RViz)                                          |
| <code>check_sim.sh</code>      | health check of a running simulation (topics, transforms, Gazebo speed)                                   |
| <code>missions/</code>         | where the Writer saves its beacons ( <code>latest.json</code> and dated copies)                           |
| <code>src/writer_robot/</code> | the package, with <code>ona/</code> inside (used by the <code>ona</code> option); the tools run from here |

### B.2 On Windows

|         |                                                                                                                                                                                                           |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Windows | Unzip <code>living_map.zip</code> into <code>C:\tsyp\</code> (right-click → Extract All → <code>C:\tsyp</code> )<br>you get <code>C:\tsyp\living_map\</code> (a short path without spaces avoids trouble) |
| Windows | <code>C:\tsyp\living_map\writer_robot\windows\install_python_deps.bat</code><br>once: <code>numpy</code> , <code>matplotlib</code> and <code>OpenCV</code> for the 2D simulators and tests                |
| Windows | <code>C:\tsyp\living_map\ona\windows\install_deps.bat</code><br>once: <code>numpy</code> and <code>pyserial</code> for the ONA                                                                            |

- The first time you run `bpsim.exe` or `bptool.exe`, Windows SmartScreen may warn (unsigned program):  
**More info → Run anyway.**

- The first `run_dashboard.bat` installs the dashboard packages by itself (needs internet once).
- When Windows asks whether Node.js or Python may use the network, allow **private networks**: otherwise the VM cannot reach the dashboard or the ONA.

## B.3 Updating later

A new `living_map.zip`: run the B.1 lines again (`unzip`, `install.sh`, `make`) and unzip it again on Windows. `install.sh` keeps every previous package and script in `~/writer_robot_backups/` and never deletes a mission. After changing a file in `~/writer_robot_ws/src/writer_robot` yourself, rebuild with the line below.

**VM terminal** `cd ~/writer_robot_ws && colcon build --packages-select writer_robot --symlink-install`  
rebuild the package by hand, then open a new terminal

# C. Check that everything works (no Gazebo)

These run in seconds to minutes and prove that the robot logic, the ONA and the protocol work before you start the heavy simulation. They are also the fastest way to show a whole mission on a laptop without the VM.

## C.1 On Windows (double-click)

**Windows** `C:\tsyp\living_map\writer_robot\windows\sim_mission_big_map.bat`  
the whole mission in the 2D simulator on the big map (Writer, then Executor), then opens a picture of both runs

**Windows** `C:\tsyp\living_map\writer_robot\windows\sim_mission_retreat_test.bat`  
the same on the small map

**Windows** `C:\tsyp\living_map\writer_robot\windows\sim_big_map.bat`  
only the Writer exploring the big map

**Windows** `C:\tsyp\living_map\writer_robot\windows\sim_retreat_test.bat`  
the Writer turning back at the purple block

**Windows** `C:\tsyp\living_map\writer_robot\windows\run_all_tests.bat`  
every robot test: 21 Writer runs, 26 missions (the Gafsa mine included), 70 node checks, 21 beacon-graph checks, 18 robot + ONA checks (15–20 minutes)

**Windows** `C:\tsyp\living_map\ona\windows\run_tests.bat`  
the ONA: 70, 20 and 3 checks (about 2 minutes)

**Windows** `C:\tsyp\living_map\beacon_net\windows\run_tests.bat`  
`C` ⇏ Python protocol cross-checks

**Windows** `C:\tsyp\living_map\beacon_net\windows\run_simulation.bat`  
one beacon-network simulation with its 13 PASS/FAIL checks

## C.2 The same in the VM

**VM terminal** `cd ~/writer_robot_ws/src/writer_robot`  
go to the package: the robot tools run from here

**VM terminal** `python3 tools/sim_mission.py --png /tmp/mission.png && xdg-open /tmp/mission.png`  
whole mission on the big map, then shows the picture

| <b>VM terminal</b>                | <code>python3 tools/sim_mission.py --world worlds/retreat_test.world --png /tmp/mission_small.png</code><br>whole mission on the small map                                                                  |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b>                | <code>python3 tools/sim2d.py --png /tmp/writer.png</code><br>the Writer only, big map, with checks                                                                                                          |
| <b>VM terminal</b>                | <code>python3 tools/run_tests.py</code><br>21 Writer runs (4 maps incl. the Gafsa mine, several seeds), PASS/FAIL per run                                                                                   |
| <b>VM terminal</b>                | <code>python3 tools/run_mission_tests.py</code><br>26 whole missions, including 4 Command Post briefings and 4 in the Gafsa mine (about 6 minutes)                                                          |
| <b>VM terminal</b>                | <code>python3 tools/test_nodes.py</code><br>the real ROS node files on a small ROS stand-in: 70 checks                                                                                                      |
| <b>VM terminal</b>                | <code>python3 tools/test_beacon_graph.py</code><br>beacon-graph routes (Dijkstra / A*): 21 checks                                                                                                           |
| <b>VM terminal</b>                | <code>python3 tools/test_ona_link.py</code><br>the robots and the real ONA together, no ROS needed: 18 checks (about 2 minutes)                                                                             |
| <b>VM terminal</b>                | <code>cd ~/living_map/ona &amp;&amp; python3 tests/test_ona.py &amp;&amp; python3 tests/test_zone.py &amp;&amp; python3 tests/test_vs_beaconnet.py</code><br>the ONA: 70, 20 and 3 checks (about 2 minutes) |
| <b>VM terminal</b>                | <code>make -C ~/living_map/beacon_net/host test</code><br>protocol cross-checks + firmware compiled for the PC + one network simulation                                                                     |
| test                              | a good run ends with                                                                                                                                                                                        |
| <code>run_tests.py</code>         | passed 21 / 21                                                                                                                                                                                              |
| <code>run_mission_tests.py</code> | passed 26 / 26                                                                                                                                                                                              |
| <code>test_nodes.py</code>        | 70 / 70 checks passed                                                                                                                                                                                       |
| <code>test_beacon_graph.py</code> | 21 / 21 checks passed                                                                                                                                                                                       |
| <code>test_ona_link.py</code>     | 18 / 18 checks passed: ALL PASSED                                                                                                                                                                           |
| ONA tests                         | 70 / 70, 20 / 20 and 3 / 3 checks passed                                                                                                                                                                    |
| <code>make test</code>            | every cross-check OK and the simulation's 13 checks PASS                                                                                                                                                    |

### C.3 Options of the 2D simulators

| tool           | option                                                              | meaning                                                 |
|----------------|---------------------------------------------------------------------|---------------------------------------------------------|
| sim2d.py       | <code>--world FILE</code>                                           | world to use (default the big map)                      |
|                | <code>--minutes M</code>                                            | simulated time limit                                    |
|                | <code>--seed N</code>                                               | random seed (sensor noise, start jitter)                |
|                | <code>--start X Y</code>                                            | start position                                          |
|                | <code>--png FILE</code>                                             | draw the run into a picture                             |
|                | <code>--quiet / --verbose / --trace</code>                          | less output / every planning decision / every step      |
|                | <code>--min-area PX</code>                                          | camera blob threshold, like <code>min_area_px</code>    |
|                | <code>--expect-turn-back</code>                                     | fail unless the robot turns back at a hazard            |
| sim_mission.py | <code>--self-hits</code>                                            | add fake LiDAR hits on the robot body (robustness test) |
|                | <code>--world, --seed, --min-area, --png, --quiet, --verbose</code> | as above                                                |

| tool        | option                                                                                                                                                                                                | meaning                                                                                                                             |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
|             | --mission FILE                                                                                                                                                                                        | skip the Writer: use a recorded mission file (e.g. from Gazebo)                                                                     |
|             | --save-mission FILE                                                                                                                                                                                   | save the Writer's mission file                                                                                                      |
|             | --frame-error M                                                                                                                                                                                       | shift the Executor's map against the Writer's (misalignment test)                                                                   |
|             | --radio-offset N                                                                                                                                                                                      | the radio starts in the middle of the record cycle                                                                                  |
|             | --briefing T:ID,ID,...[:stay]                                                                                                                                                                         | a Command Post briefing at simulated second T (targets in order; stay = do not return to the exit); T:abort calls the Executor back |
|             | --wait-briefing S                                                                                                                                                                                     | the Executor waits S seconds at the exit for a briefing                                                                             |
| VM terminal | python3 tools/sim_mission.py --mission ~/writer_robot_ws/missions/latest.json --png /tmp/replay.png<br>replay a mission recorded in Gazebo in the 2D Executor, in seconds                             |                                                                                                                                     |
| VM terminal | python3 tools/sim_mission.py --mission missions/demo_big.json --wait-briefing 30 --briefing 5:25,11,20<br>a briefing: the Executor treats #25, #11, #20 in that order, then RESULT: ALL CHECKS PASSED |                                                                                                                                     |
| VM terminal | python3 tools/sim_mission.py --mission missions/demo_big.json --briefing 40:20,30:stay<br>re-targeted during the mission; it stays at its last target                                                 |                                                                                                                                     |
| VM terminal | python3 tools/sim_mission.py --mission missions/demo_big.json --briefing 45:abort<br>called back: it stops and follows the beacons out                                                                |                                                                                                                                     |

## D. The Command Post dashboard

### D.1 Start it

|               |                                                                                                                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Windows       | C:\tsyp\living_map\beacon_net\windows\run_dashboard.bat<br>recommended: the dashboard on Windows (installs its packages the first time). Keep the window open.                                                        |
| Browser       | http://localhost:3000<br>open the dashboard in the Windows browser                                                                                                                                                    |
| Windows       | C:\tsyp\living_map\beacon_net\windows\run_dashboard_demo.bat<br>dashboard + a scripted mission with no robot and no simulator: votes, both robots on their circles, a lying gateway, a SLAM drift, the satellite link |
| VM terminal   | cd ~/living_map/command_post && npm start<br>alternative: the dashboard inside the VM                                                                                                                                 |
| VM terminal   | firefox http://localhost:3000 &<br>open it in the VM                                                                                                                                                                  |
| VM terminal   | cd ~/living_map/command_post && npm run demo<br>VM: dashboard + scripted mission                                                                                                                                      |
| VM or Windows | npm run mock<br>(in command_post/) only the scripted feeder, against a dashboard already running                                                                                                                      |
| VM or Windows | ONA=0 npm run demo<br>VM syntax: the scripted mission with one gateway and no ONA                                                                                                                                     |

|               |                                                                                                                                           |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| VM or Windows | TARGET=http://192.168.1.20:3000 node mock_feeder.js<br>VM syntax: scripted feeder against another machine (Windows: set TARGET=... first) |
| VM or Windows | PORT=4000 GATEWAYS=gw1,gw2,gw3 npm start<br>VM syntax: another port and the list of expected gateways                                     |

## D.2 Where the dashboard is, from each place

| dashboard runs on | from the Windows browser                                                 | from programs in the VM | from another laptop           |
|-------------------|--------------------------------------------------------------------------|-------------------------|-------------------------------|
| Windows           | http://localhost:3000                                                    | http://10.0.2.2:3000    | http://<Windows LAN IP>:3000  |
| the VM            | not reachable with plain NAT (add a VirtualBox port forward 3000 → 3000) | http://localhost:3000   | through the same port forward |

## D.3 Using it

- **Pins** are beacons, coloured by kind; they fade as each record's confidence decays. Dashed arrows are the next links (the way out). Click a pin: severity, confidence now and at creation, age, accuracy, hops, signal, its vote, and its whole route to the exit in green with the position-error circle.
- **Votes**: ✓3/3 or ✓2/3 = confirmed by two or three gateways. A dotted, faded pin marked 1/3 was reported by one gateway only: it is not counted and never replaces a confirmed version.
- **Lost beacons**: red dashed ring and "LOST?". Hollow pins: aged-out events kept as route markers. Grey: retracted.
- **Gateways and robots**: three antennas around the building; around them, each one's measured distance to a robot as a circle; the robot where they cross, inside its error ellipse; a small dashed circle where the robot thinks it is. **Spheres** toggles the circles; **Follow** keeps a robot in view.
- **Outside network area panel**: uplink (LTE / SATELLITE / NO UPLINK), each gateway's agreement (IGNORED after 3 strikes), records confirmed / waiting / conflicts, queue, frame check (holds / DRIFTED), briefing and its acknowledgement. **Executor** and **Writer** panels: measured by N gateways ± error, own SLAM agrees or DRIFT, phase, battery, mission progress.
- **Top bar**: one chip per gateway, records held, △ HMAC rejects. **Feed**: every event and ONA alert.
- **Mission planner**: *Select on map*, click 2–3 pins, reorder, *Dispatch mission* (untick "back to the exit afterwards" to keep the Executor there). Or open a pin and press *Executor path to here*. *Call the Executor back* sends a signed ABORT.
- Base maps Dark / Satellite / Street / Offline grid (automatic without internet).
- Restarting the dashboard (Ctrl-C in its window, then start it again) clears every pin: do it between a simulator demo and a robot demo so the two do not mix.

## D.4 Check the server from a terminal

|             |                                                                                                                                                                                 |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VM terminal | curl -s http://10.0.2.2:3000/api/health<br>from the VM to a dashboard on Windows: {"ok":true,...} proves the network path works (use localhost if the dashboard runs in the VM) |
| VM terminal | curl -s http://10.0.2.2:3000/api/state   head -c 600<br>the start of the stored state                                                                                           |

|                    |                                                                                                                                                                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>curl -s -X POST http://10.0.2.2:3000/api/beacon -H "Content-Type: application/json" -d '{"beacon_id":99,"event_type":"gas","value":50,"lat":36.8066,"lon":10.1816,"ttl":20, "gateway_id":"test"}'</pre> <p>post one test beacon: a gas pin appears at once</p> |
| <b>Windows</b>     | <pre>curl.exe -s http://localhost:3000/api/beacons</pre> <p>Windows: the last 200 beacons as JSON</p>                                                                                                                                                               |
| <b>Browser</b>     | <pre>http://localhost:3000/api/mission-dispatch/latest</pre> <p>the last mission dispatched from the planner</p>                                                                                                                                                    |

## E. The Writer robot

### E.1 Start it

|                    |                                                                                                                                                                                                                                                                |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh</pre> <p>big map (default): Gazebo + robot + SLAM + vision + beacons + mission recorder + explorer + RViz</p>                                                                                                           |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh headless</pre> <p>the same without the Gazebo window (much faster; watch RViz)</p>                                                                                                                                      |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh retreat</pre> <p>the small map: a purple block after a corner, the robot must turn back (quick, good for a first test)</p>                                                                                              |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh small</pre> <p>the original small arena</p>                                                                                                                                                                             |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh headless ona</pre> <p>with the Outside Network Area: every beacon reaches the dashboard through three gateways as it is dropped, and the Writer appears on its distance circles (start the dashboard first, part D)</p> |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh retreat headless ona</pre> <p>options combine in any order</p>                                                                                                                                                          |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh mine headless ona</pre> <p>the Gafsa mine (part N)</p>                                                                                                                                                                  |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh headless gpu</pre> <p>try the graphics card (VirtualBox 3D acceleration ON, see J.1); if Gazebo does not start, drop gpu</p>                                                                                            |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh headless slow</pre> <p>slower robot (0.5 m/s, slower turns), in case it bumps into things on your computer</p>                                                                                                          |
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh ekf</pre> <p>experiment only: odometry from the wheels + IMU (EKF) instead of Gazebo's true motion (the default)</p>                                                                                                    |

The script prints == [0/5] cleanup == ... == [5/5] RViz ==, waits for /scan, brings SLAM up (configure, activate) and waits for the map→odom transform before the robot moves. That takes 30–90 s in a VM. Leave this terminal alone; Ctrl-C in it stops everything.

### E.2 Watch what it does (new terminals)

|                    |                                                                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>tail -f /tmp/explorer.log</pre> <p>the robot's decisions: targets, RETREAT, LOOK, recoveries, EXPLORATION COMPLETE, DONE</p>                                            |
| <b>VM terminal</b> | <pre>grep -E 'time so far total time recover' /tmp/explorer.log</pre> <p>where the time goes, one line per simulated minute: driving, camera looks, planning, recovering</p> |

|                    |                                                                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>bash ~/writer_robot_ws/check_sim.sh</code><br>health check: rate of every topic, the odometry and SLAM transforms, Gazebo speed, last log lines.<br>Send its output if something does not work. |
| <b>VM terminal</b> | <code>ros2 topic echo /writer/explorer_status</code><br>the state and what it is doing (Ctrl-C to stop)                                                                                               |
| <b>VM terminal</b> | <code>grep health /tmp/vision.log</code><br>camera health every 10 s: colour frames, depth frames, hazard frames per colour (M.2)                                                                     |
| <b>VM terminal</b> | <code>tail -f /tmp/vision.log</code><br>every VISION EVENT (a hazard that gets a beacon)                                                                                                              |
| <b>VM terminal</b> | <code>tail -f /tmp/beacon.log</code><br>every beacon dropped, with its link                                                                                                                           |
| <b>VM terminal</b> | <code>tail -f /tmp/mission.log</code><br>every record written to the mission file (kind + frame)                                                                                                      |
| <b>VM terminal</b> | <code>ros2 topic echo /writer/beacon_dropped</code><br>the beacons as JSON (id, type, x, y, link_id, hazard x/y)                                                                                      |
| <b>VM terminal</b> | <code>ros2 topic echo /writer/hazard</code><br>every camera sighting (type, map x/y, range)                                                                                                           |
| <b>VM terminal</b> | <code>ros2 topic echo /outside_network/beacon_gps</code><br>the beacons in lat/lon                                                                                                                    |
| <b>VM terminal</b> | <code>tail -f /tmp/ona.log</code><br>option ona: the ONA (uplink, queue, each gateway, records confirmed, the robot located), a line every 5 s                                                        |

A normal run reads like this in `/tmp/explorer.log` (small map):

```
map and pose received - exploring
first a full turn on the spot so SLAM maps the surroundings
new hazard: radiation at (6.75, 2.99)
RETREAT: radiation ahead (2.8 m) -> returning the way it came (1.6 m)
retreat finished - exploring the rest of the map
no reachable frontier and nothing left for the camera - EXPLORATION COMPLETE
DONE - robot stopped
total time 1.0 min: driving 27 s, camera looks 13 s, planning 5 s, retreating 6 s, going home 8 s; driven 18 m
```

In RViz the map grows; the blue line is the planned path; coloured discs are hazard keep-out zones; each beacon is a post with a ball on top (green EXIT, blue trail, magenta radiation, orange fire, yellow gas) and its id, with light-blue lines to its next beacon; the state is written above the robot.

### E.3 Stop it and keep the mission

|                    |                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>Ctrl-C</code> (in the <code>run_explorer.sh</code> terminal)<br>stops everything the script started, once the log says DONE                   |
| <b>VM terminal</b> | <code>bash ~/writer_robot_ws/stop_demo.sh</code><br>from any terminal: kills every simulation process (if a window hangs, or before starting again) |
| <b>VM terminal</b> | <code>ls -l ~/writer_robot_ws/missions/</code><br><code>latest.json</code> (always the last run) and <code>mission_&lt;date&gt;.json</code> copies  |
| <b>VM terminal</b> | <code>python3 -m json.tool ~/writer_robot_ws/missions/latest.json   less</code><br>read the mission file (q to quit)                                |

```
VM terminal grep -c '"frame"' ~/writer_robot_ws/missions/latest.json
how many beacons were recorded
```

```
VM terminal cp ~/writer_robot_ws/missions/latest.json ~/writer_robot_ws/missions/good_run.json
keep a good run under its own name (the next Writer run overwrites latest.json)
```

## E.4 Change a parameter

Parameters are read when a node starts. To change one, stop that node and start it again with `-p`, while the rest keeps running.

```
VM terminal pkill -f frontier_explorer; ros2 run writer_robot frontier_explorer --ros-args -p
use_sim_time:=true -p v_max:=0.3 -p verbose:=true > /tmp/explorer.log 2>&1 &
restart the explorer slower, logging every planning decision. It re-plans from where the robot is.
```

```
VM terminal pkill -f vision_event_detector; ros2 run writer_robot vision_event_detector
--ros-args -p use_sim_time:=true -p min_area_px:=400 > /tmp/vision.log 2>&1 &
restart the camera node with a smaller blob threshold (sees blocks from farther away)
```

```
VM terminal ros2 param list /frontier_explorer
all parameters of a running node
```

```
VM terminal ros2 param get /frontier_explorer keepout_radius
the value a node is using
```

| node                  | useful parameters (default)                                                                                                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| frontier_explorer     | v_max 0.70 · w_max 1.6 · look_w_max 1.6 · stuck_time 8 · keepout_radius 1.2 · coverage true · camera_range 3.2 · robot_radius 0.36 · retreat_min 1.5 · retreat_clear 2.5 · home_on_finish true · verbose false |
| vision_event_detector | min_area_px 800 · drop_range_m 4.0 · dedup_radius_m 1.5 · camera_height_m 0.135 · hazard_width_m 0.30                                                                                                          |
| beacon_drop_node      | turn_threshold_deg 35 · max_trail_gap_m 2.5 · trail_dedup_m 1.2 · pass_radius_m 0.5 · magazine_capacity 120 · exit_beacon true                                                                                 |
| mission_recorder      | mission_dir · anchor_lat 36.8065 · anchor_lon 10.1815 · key demo · net_id 42                                                                                                                                   |
| ona_link              | robot writer/executor · world_file · ona_config · ona_host 127.0.0.1 · ona_port 47100 · ping_period 2.0                                                                                                        |

The full list, with every default read from the code, is in the Architecture document (sections 5.7, 6.3, 11.6).

## E.5 Start the Writer by hand, one terminal per piece (advanced)

This is what `run_explorer.sh` does. Use it to restart a single piece or to see its output live. Type the lines in this order, each in a new terminal, waiting for each to be up.

```
VM terminal bash ~/writer_robot_ws/stop_demo.sh
T0: clean start
```

```
VM terminal ros2 launch writer_robot gazebo_sim.launch.py
T1: Gazebo, robot, bridges (incl. the odometry TF). Add headless:=true for no window,
world:=<file> for another map.
```

```
VM terminal ros2 topic hz /scan
check: about 8 Hz. Ctrl-C, then continue.
```

```
VM terminal ros2 run slam_toolbox async_slam_toolbox_node --ros-args --params-file $(ros2 pkg
prefix writer_robot)/share/writer_robot/config/slam_toolbox_params.yaml -p
use_sim_time:=true
T2: SLAM (inactive until the next two lines)
```

|                    |                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>ros2 lifecycle set /slam_toolbox configure</code><br>T3: then...                                                                                    |
| <b>VM terminal</b> | <code>ros2 lifecycle set /slam_toolbox activate</code><br>...SLAM starts publishing /map and map→odom                                                     |
| <b>VM terminal</b> | <code>ros2 run writer_robot vision_event_detector --ros-args -p use_sim_time:=true</code><br>T3: camera                                                   |
| <b>VM terminal</b> | <code>ros2 run writer_robot beacon_drop_node --ros-args -p use_sim_time:=true</code><br>T4: beacons                                                       |
| <b>VM terminal</b> | <code>ros2 run writer_robot mission_recorder --ros-args -p use_sim_time:=true</code><br>T5: mission file                                                  |
| <b>VM terminal</b> | <code>ros2 run writer_robot frontier_explorer --ros-args -p use_sim_time:=true</code><br>T6: the brain (the robot starts moving)                          |
| <b>VM terminal</b> | <code>rviz2 -d \$(ros2 pkg prefix writer_robot)/share/writer_robot/config/writer.rviz</code><br><code>--ros-args -p use_sim_time:=true</code><br>T7: RViz |

## E.6 Drive by hand and test the trapdoor (optional)

|                    |                                                                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>ros2 run teleop_twist_keyboard teleop_twist_keyboard</code><br>drive with the keyboard (i, j, l, comma, k). Do not run it together with the explorer. |
| <b>VM terminal</b> | <code>ros2 topic pub -1 /trapdoor_cmd std_msgs/msg/Float64 "{data: -1.2}"</code><br>open the beacon trapdoor                                                |
| <b>VM terminal</b> | <code>ros2 topic pub -1 /trapdoor_cmd std_msgs/msg/Float64 "{data: 0.0}"</code><br>close it                                                                 |

# F. The Executor robot

## F.1 Start it

The Executor needs a mission file: the one your Writer just recorded, or a ready-made one. Stop the Writer first (Ctrl-C): one robot runs at a time.

|                    |                                                                                                                                                                                                  |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>~/writer_robot_ws/run_executor.sh</code><br>uses <code>~/writer_robot_ws/missions/latest.json</code> (your last Writer run). The world is rebuilt with a beacon model at every drop point. |
| <b>VM terminal</b> | <code>~/writer_robot_ws/run_executor.sh demo</code><br>ready-made mission on the big map (43 beacons, 4 hazards): no Writer run needed                                                           |
| <b>VM terminal</b> | <code>~/writer_robot_ws/run_executor.sh demo retreat</code><br>ready-made mission on the small map (about 1 minute)                                                                              |
| <b>VM terminal</b> | <code>~/writer_robot_ws/run_executor.sh ~/writer_robot_ws/missions/good_run.json</code><br>any saved mission file                                                                                |
| <b>VM terminal</b> | <code>~/writer_robot_ws/run_executor.sh demo headless</code><br>no Gazebo window (faster); add <code>headless</code> to any of the above                                                         |
| <b>VM terminal</b> | <code>~/writer_robot_ws/run_executor.sh demo headless ona</code><br>with the Outside Network Area: it waits at the exit for a briefing from the dashboard (F.3)                                  |

**VM terminal** `~/writer_robot_ws/run_executor.sh mine headless ona`  
the ready-made Gafsa mine mission (part N)

**VM terminal** `~/writer_robot_ws/run_executor.sh demo headless slow`  
slower robot (0.5 m/s)

**VM terminal** `~/writer_robot_ws/run_executor.sh demo ekf`  
experiment only: wheel + IMU odometry (EKF)

## F.2 Watch it (new terminals)

**VM terminal** `tail -f /tmp/executor.log`  
the mission: records received, briefing, targets, GOTO, approaching, TREATED, RETURN, MISSION COMPLETE, and a "time so far" line every simulated minute

**VM terminal** `cat /tmp/exec_radio.log`  
the LoRa receiver: how many frames were verified and decoded

**VM terminal** `grep -E "skipped|approach failed|unreachable|attempt|treating from here" /tmp/executor.log`  
why a hazard was not treated at the first try (and what the robot did about it)

**VM terminal** `ros2 topic echo /executor/status`  
phase and progress

**VM terminal** `ros2 topic echo /executor/mission_events`  
one JSON message per treated hazard

**VM terminal** `ros2 topic echo /executor/lora_rx --once`  
one decoded beacon record as the robot receives it

**VM terminal** `grep health /tmp/exec_vision.log`  
the Executor's camera health

```
[executor] 5 beacon records (3 trail, 2 other), exit #0; mission: 1 target(s) RADIATION #4
[executor] GOTO RADIATION #4: 5 beacons, 6.0 m along the beacon chain
[executor] at beacon #4: approaching the radiation to 1.6 m
[executor] camera confirms RADIATION #4 (0.02 m from its record)
[executor] RADIATION #4 TREATED (placing radiation shielding, from 1.90 m, confirmed by camera)
[executor] RETURN to the exit: following 5 "next" hops
[executor] MISSION COMPLETE: 1/1 targets treated, back at the exit
```

(demo `retreat`; on the big map there are 43 records and 4 targets, each with its own GOTO ... TREATED.) In RViz: the beacons and their links; a green line through the beacons still to visit; an orange line for the current leg; each hazard zone turns green when treated. In Gazebo a green disc appears around every treated hazard.

## F.3 Brief it from the Command Post (option `ona`)

With `ona`, the Executor waits 60 s of simulated time at the exit for a briefing. Start the dashboard first (part D).

**VM terminal** `~/writer_robot_ws/run_executor.sh headless ona`  
the mission the Writer recorded (`run_executor.sh demo headless ona` for the ready-made one)

**Browser** (dashboard) Select on map → click 2–3 hazards → Dispatch mission  
the ONA signs it, the gateways transmit it, the robot checks it: "✓ acknowledged" in the ONA panel within a few seconds; it treats them in your order. Untick "back to the exit afterwards" to keep it there.

|                    |                                                                                                                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>grep -iE "briefing TREATED COMPLETE" /tmp/executor.log</code><br>what the Executor did with the briefing                                                                                                               |
| <b>VM terminal</b> | <code>ros2 topic echo /executor/briefing</code><br>the briefing as the robot received it (latched)                                                                                                                           |
| <b>VM terminal</b> | <code>ros2 topic echo /executor/telemetry</code><br>phase, mission id, acknowledgement, done / total: what goes into its position reports                                                                                    |
| <b>VM terminal</b> | <code>ros2 topic echo /ona_link/status</code><br>the robot's radio: pings sent, records, briefings, simulation speed                                                                                                         |
| <b>Browser</b>     | (dashboard) Call the Executor back<br>a signed ABORT: it stops and follows the beacons out                                                                                                                                   |
| <b>VM terminal</b> | <code>WAIT_BRIEFING=180 ~/writer_robot_ws/run_executor.sh demo headless ona</code><br>more time to dispatch (simulated seconds; Gazebo in a VM is slower than real time). A briefing that arrives later still re-targets it. |

## F.4 Change what it does

|                    |                                                                                                                                                                                                                                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>kill -f executor_node; ros2 run writer_robot executor_node --ros-args -p use_sim_time:=true -p target_kinds:=RADIATION,GAS -p treat_time:=10.0 &gt; /tmp/executor.log 2&gt;&amp;1 &amp;</code><br>restart only the Executor brain: treat only radiation and gas, 10 s each (it rebuilds the tree from the radio in a few seconds) |
| <b>VM terminal</b> | <code>kill -f executor_node; ros2 run writer_robot executor_node --ros-args -p use_sim_time:=true -p target_ids:='"13,30"' &gt; /tmp/executor.log 2&gt;&amp;1 &amp;</code><br>only beacons 13 and 30                                                                                                                                    |

| parameter                                    | default                       | meaning                                                                       |
|----------------------------------------------|-------------------------------|-------------------------------------------------------------------------------|
| target_kinds                                 | RADIATION,THERMAL,GAS, VICTIM | what to treat (comma-separated, no spaces on the command line)                |
| target_ids                                   | (all)                         | only these beacon ids                                                         |
| wait_briefing                                | 0 (60 with ona)               | seconds to wait at the exit for a Command Post briefing                       |
| standoff / victim_standoff                   | 1.6 / 0.8                     | distance from the hazard while treating (m)                                   |
| treat_time                                   | 4.0                           | seconds per target                                                            |
| treat_max                                    | 3.0                           | a hazard in plain sight this close can be treated from where the robot is (m) |
| retry_skipped                                | true                          | one more attempt at every skipped hazard before going home                    |
| min_confidence                               | 0.15                          | skip hazards that have faded below this (a briefed hazard is always taken)    |
| age_mode                                     | mission                       | wallclock = age records by the real clock (gas fades in minutes)              |
| keepout_radius, v_max                        | 1.2, 0.70                     | as for the Writer (slow: 0.5 m/s)                                             |
| beacon_radio_sim: mission_file, key, rate_hz | latest.json, demo, 20         | the receiver                                                                  |

## F.5 Beacon-graph navigation (Dijkstra / A\*): ready for later

`writer_robot/beacon_graph_nav.py` treats the beacons as the points of a graph (tree links + wall-free shortcuts) and finds routes with Dijkstra or A\*. The Executor does not use it yet; the end of the file lists the five steps to plug it in. Try it without ROS (in `~/writer_robot_ws/src/writer_robot`):

|               |                                                                                                                                                                                     |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VM or Windows | <code>python3 tools/beacon_graph_demo.py --png /tmp/graph.png</code><br>routes from the EXIT to every hazard: tree, Dijkstra and A*, plus a picture                                 |
| VM or Windows | <code>python3 tools/beacon_graph_demo.py --mission ~/writer_robot_ws/missions/latest.json --png /tmp/mine.png</code><br>the same on the mission your Writer just recorded in Gazebo |
| VM or Windows | <code>python3 tools/beacon_graph_demo.py --link-radius 0</code><br>tree only (no shortcuts), for comparison                                                                         |
| Windows       | <code>C:\tsyp\living_map\writer_robot\windows\beacon_graph_demo.bat</code><br>Windows: the demo with its picture                                                                    |

## F.6 Start the Executor by hand (advanced)

|             |                                                                                                                                                                                                                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VM terminal | <code>ros2 run writer_robot mission_world --mission ~/writer_robot_ws/missions/latest.json --out /tmp/executor_world.world</code><br>T1: build the world with the beacons in place                                                                                                                                      |
| VM terminal | <code>ros2 launch writer_robot gazebo_sim.launch.py world:=/tmp/executor_world.world robot:=executor</code><br>T1: Gazebo with the blue Executor                                                                                                                                                                        |
| VM terminal | <code>ros2 run slam_toolbox async_slam_toolbox_node --ros-args --params-file \$(ros2 pkg prefix writer_robot)/share/writer_robot/config/slam_toolbox_params.yaml -p use_sim_time:=true</code><br>T2: SLAM; then <code>ros2 lifecycle set /slam_toolbox configure</code> and ... activate in T3                          |
| VM terminal | <code>ros2 run writer_robot beacon_radio_sim --ros-args -p use_sim_time:=true -p mission_file:=\$HOME/writer_robot_ws/missions/latest.json</code><br>T3: the LoRa receiver                                                                                                                                              |
| VM terminal | <code>ros2 run writer_robot vision_event_detector --ros-args -p use_sim_time:=true -r /writer/hazard:=/executor/hazard -r /writer/events:=/executor/vision_events -r /writer/vision_target:=/executor/vision_target</code><br>T4: the Executor's camera (topics renamed)                                                |
| VM terminal | <code>ros2 run writer_robot executor_node --ros-args -p use_sim_time:=true</code><br>T5: the brain                                                                                                                                                                                                                      |
| VM terminal | <code>rviz2 -d \$(ros2 pkg prefix writer_robot)/share/writer_robot/config/executor.rviz --ros-args -p use_sim_time:=true</code><br>T6: RViz                                                                                                                                                                             |
| VM terminal | <code>cd ~/writer_robot_ws/src/writer_robot/ona &amp;&amp; python3 -m ona --config ona_config_gazebo_big.json --udp 127.0.0.1:47100 --command-post http://10.0.2.2:3000</code><br>T7 (optional): the ONA                                                                                                                |
| VM terminal | <code>ros2 run writer_robot ona_link --ros-args -p use_sim_time:=true -p robot:=executor -p world_file:=/tmp/executor_world.world -p mission_file:=\$HOME/writer_robot_ws/missions/latest.json</code><br>T8 (optional): the robot's radio to the three gateways; then start T5 with <code>-p wait_briefing:=60.0</code> |

## G. The Outside Network Area (three gateways)

The ONA is the only bridge between the zone and the outside world. Three gateways listen around the building; the ONA accepts a record only when two of them heard the identical signed bytes, locates the robots from the gateways' distance measurements (like GPS), translates everything to GPS and forwards it to the Command Post over LTE, or over satellite when LTE fails. It also signs and sends the Command Post's briefings to the Executor. It is one program (`python -m ona`) that runs in the VM or on Windows; its inputs are real gateways on USB, robots in Gazebo, the zone simulator or the beacon simulator.

**WATCH OUT** Start the dashboard first (part D). Run **one ONA at a time**: two would both post to the dashboard, and the second cannot open UDP port 47100.

### G.1 The zone simulator: a whole mission, three gateways

No ROS, no Gazebo: a built-in building with 16 beacons, three gateways and an Executor, as the gateways hear it. Two windows: the ONA, then the zone.

|                    |                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Windows</b>     | <code>C:\tsyp\living_map\ona\windows\run_ona.bat</code><br>window 1: the ONA, listening on UDP 47100, posting to <code>http://localhost:3000</code>                                                                                          |
| <b>Windows</b>     | <code>C:\tsyp\living_map\ona\windows\simulate_zone.bat</code><br>window 2: the mission. On the dashboard: select 2-3 hazards and <b>Dispatch mission</b> within a minute; the Executor acknowledges and treats them in that order.           |
| <b>Windows</b>     | <code>C:\tsyp\living_map\ona\windows\simulate_faults.bat</code><br>instead of window 2: gw3 lies (IGNORED), gw2 forges a victim at 60 s (stays 1/3), beacon 6 is crushed at 90 s (LOST?), the robot's SLAM slips 2.9 m at 150 s (SLAM DRIFT) |
| <b>Windows</b>     | <code>C:\tsyp\living_map\ona\windows\run_ona.bat --lte-outage 60-150</code><br>instead of window 1 (in cmd): LTE down from 60 s to 150 s: SATELLITE on the dashboard, only the essentials go out                                             |
| <b>VM terminal</b> | <code>cd ~/living_map/ona &amp;&amp; ./run_ona.sh --command-post http://10.0.2.2:3000</code><br>the same ONA in the VM (dashboard on Windows)                                                                                                |
| <b>VM terminal</b> | <code>cd ~/living_map/ona &amp;&amp; ./simulate_zone.sh</code><br>second VM terminal: the nominal zone                                                                                                                                       |
| <b>VM terminal</b> | <code>cd ~/living_map/ona &amp;&amp; ./simulate_zone.sh --liar gw3 --forge gw2@60 --kill 6@90 --slam-slip 150:2.5,-1.5</code><br>or the zone with every fault                                                                                |

### G.2 The beacon network simulator through the vote

|                    |                                                                                                                                                                                                                                                                       |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Windows</b>     | <code>C:\tsyp\living_map\ona\windows\bpsim_to_ona.bat</code><br>the beacons' own C code, 14 beacons, three gateways, 10× real time, through the ONA to the dashboard (about 6 minutes): every record ✓3/3, beacon 6 LOST at minute 25, the attacker's frames rejected |
| <b>VM terminal</b> | <code>cd ~/living_map/ona &amp;&amp; ../beacon_net/host/bpsim --gateways 3 --speed 20 --quiet   python3 -m ona --stdin --config ona_config_bpsim.json --command-post http://10.0.2.2:3000</code><br>the same in the VM                                                |

## G.3 With the Gazebo robots

The `ona` option of both run scripts starts, in the VM, the ONA (posting to the dashboard on Windows) and the robot's `ona_link` node, which plays the three gateways' radio. Commands are in E.1 and F.3; variants:

|                    |                                                                                                                                                                                                                                                |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>COMMAND_POST=http://192.168.1.20:3000 ~/writer_robot_ws/run_executor.sh demo headless ona</code><br>a dashboard on another computer                                                                                                      |
| <b>Windows</b>     | <code>C:\tsyp\living_map\ona\windows\run_ona_for_gazebo.bat</code><br>the ONA on Windows instead of the VM (big map; add <code>--config ona_config_gazebo_retreat.json</code> for the small one). Allow Python on private networks when asked. |
| <b>VM terminal</b> | <code>ONA_HOST=10.0.2.2 ~/writer_robot_ws/run_executor.sh demo headless ona</code><br>then the robot sends its radio lines to the Windows ONA (works the same with <code>run_explorer.sh</code> )                                              |
| <b>VM terminal</b> | <code>bash ~/writer_robot_ws/stop_demo.sh</code><br>stops everything, the ONA and <code>ona_link</code> included                                                                                                                               |

## G.4 Look inside

|                      |                                                                                                                                                                                                                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b>   | <code>tail -f /tmp/ona.log</code><br>the ONA started by the <code>ona</code> option: link, queue, each gateway, records confirmed, robots                                                                                                                                           |
| <b>VM terminal</b>   | <code>cat /tmp/ona_link.log</code><br>the robot's radio node: gateways, walls, briefings received                                                                                                                                                                                   |
| <b>VM terminal</b>   | <code>python3 -c "import sqlite3; [print(r) for r in sqlite3.connect('/tmp/ona_queue.sqlite3').execute('select ts, event, detail_json from audit order by id desc limit 15')]"</code><br>the last 15 decisions of the ONA (rejected frames, votes, alerts, link changes, briefings) |
| <b>VM or Windows</b> | <code>python3 -m ona --help</code><br>(in <code>ona/</code> ) every option                                                                                                                                                                                                          |

| <code>python3 -m ona</code> option              | meaning                                                                                                                                                                                                              |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--config FILE</code>                      | gateways (positions), anchor, key: <code>ona_config.json</code> (zone simulator), <code>ona_config_gazebo_big.json</code> / <code>_retreat.json</code> (Gazebo), <code>ona_config_bpsim.json</code> , or yours (K.5) |
| <code>--serial gw1=COM3 (x3)</code>             | a real gateway board on a serial port                                                                                                                                                                                |
| <code>--udp HOST:PORT</code>                    | gateway lines over UDP (robots in Gazebo, the zone simulator); 0.0.0.0:47100 by default in the scripts                                                                                                               |
| <code>--stdin / --exec CMD</code>               | gateway lines from a pipe (bpsim) or from a command                                                                                                                                                                  |
| <code>--command-post URL</code>                 | the dashboard (default from the config, else <code>http://localhost:3000</code> )                                                                                                                                    |
| <code>--db FILE · --fresh</code>                | queue and audit file · start with an empty queue                                                                                                                                                                     |
| <code>--lte-outage 60-150,300-320</code>        | simulated LTE outages (seconds after start)                                                                                                                                                                          |
| <code>--no-lte · --no-sat</code>                | satellite only · no satellite                                                                                                                                                                                        |
| <code>--strict</code>                           | never show unconfirmed (1/3) records                                                                                                                                                                                 |
| <code>--quiet</code>                            | less output                                                                                                                                                                                                          |
| zone simulator option                           | meaning                                                                                                                                                                                                              |
| <code>--speed S · --minutes M · --seed N</code> | simulated seconds per real second (2 in the scripts) · duration · random seed                                                                                                                                        |
| <code>--robot executor writer</code>            | which robot the gateways follow                                                                                                                                                                                      |
| <code>--briefing-wait S · --no-wait</code>      | how long the Executor waits for a briefing (120 s) · go at once                                                                                                                                                      |

| zone simulator option                | meaning                                                                  |
|--------------------------------------|--------------------------------------------------------------------------|
| --liar gw3                           | that gateway re-signs changed records                                    |
| --forge gw2@60                       | that gateway reports a forged victim at 60 s                             |
| --corrupt gw1 · --gw-down gw2@60-140 | damaged frames from a gateway · a gateway off for a while                |
| --kill 6@90                          | beacon 6 destroyed at 90 s                                               |
| --slam-slip 150:2.5,-1.5             | the robot's own map jumps by (2.5, -1.5) m at 150 s                      |
| --mission FILE --world FILE          | a recorded mission and its Gazebo world instead of the built-in building |

## H. The beacon network (LMB2)

`bpsim` runs the real beacon firmware logic for every beacon on a simulated 868 MHz channel (walls, rubble, collisions, an attacker) and checks 13 properties at the end. `bptool` builds, signs and decodes single frames.

### H.1 Run the network simulator

|                    |                                                                                                                                                                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>cd ~/living_map/beacon_net/host</code><br>the simulator folder                                                                                                                                                               |
| <b>VM terminal</b> | <code>./bpsim &gt; /dev/null</code><br>60 simulated minutes, 14 beacons, in a few seconds. The timeline and the PASS/FAIL checks print on the screen; <code>&gt; /dev/null</code> hides the JSON lines meant for a gateway bridge. |
| <b>VM terminal</b> | <code>./bpsim --beacons 30 --minutes 90 --rubble 2.5 &gt; /dev/null</code><br>a bigger, harsher network                                                                                                                            |
| <b>VM terminal</b> | <code>./bpsim --kill 3@20 --gw-late 40 &gt; /dev/null</code><br>destroy beacon 3 at minute 20, switch the gateway on at minute 40 (it must pull everything)                                                                        |
| <b>VM terminal</b> | <code>./bpsim --sf 12 --minutes 120 &gt; /dev/null</code><br>long range, slow radio (SF12)                                                                                                                                         |
| <b>VM terminal</b> | <code>./bpsim --speed 20   head -5</code><br>what the gateway prints: JSON lines                                                                                                                                                   |
| <b>VM terminal</b> | <code>./bpsim --gateways 3 --speed 20   head -5</code><br>three gateways: one JSON line per gateway (" <code>gw</code> ": " <code>gw1</code> ", ...), what the ONA reads (G.2)                                                     |
| <b>Windows</b>     | <code>cd /d C:\tsyp\living_map\beacon_net\windows</code><br>Windows: the prebuilt .exe folder (in cmd)                                                                                                                             |
| <b>Windows</b>     | <code>bpsim.exe --beacons 30 --rubble 2.5 &gt; nul</code><br>the same simulator on Windows ( <code>run_simulation.bat</code> = no options)                                                                                         |

| bpsim option  | meaning (default)                                          |
|---------------|------------------------------------------------------------|
| --beacons N   | beacons in the chain (14, max 38)                          |
| --minutes M   | simulated duration (60)                                    |
| --speed S     | 0 = as fast as possible, 1 = real time, 20 = 20× (0)       |
| --sf 7..12    | LoRa spreading factor (9)                                  |
| --round S     | gossip round in seconds (0 = derived from SF: 96 s at SF9) |
| --digest S    | gateway pull period, 0 = off (90)                          |
| --kill ID@MIN | destroy beacon ID at minute MIN (6@25; 0@0 = none)         |
| --gateways N  | number of gateways around the building (1); 3 for the ONA  |

| bpsim option       | meaning (default)                                                    |
|--------------------|----------------------------------------------------------------------|
| --no-attacker      | no forged, tampered or replayed frames                               |
| --no-seed          | do not prime new beacons with the route (to see why seeding matters) |
| --gw-late MIN      | gateway powered on late                                              |
| --rubble DB        | extra attenuation per metre of debris (1.2)                          |
| --seed N · --quiet | random seed (7) · no timeline                                        |

## H.2 Tests and stress matrix

|                      |                                                                                                                                                              |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b>   | make -C ~/living_map/beacon_net/host<br>build bpsim and bptool                                                                                               |
| <b>VM terminal</b>   | make -C ~/living_map/beacon_net/host test<br>firmware compiled for the PC + C/Python/firmware cross-checks + one simulation                                  |
| <b>VM terminal</b>   | make -C ~/living_map/beacon_net/host matrix<br>the 42-scenario stress matrix (about 20 s): must end with <b>42/42 PASS</b>                                   |
| <b>VM terminal</b>   | make -C ~/living_map/beacon_net/host windows<br>rebuild the Windows .exe files (needs the MinGW cross-compiler: <code>sudo apt install -y mingw-w64</code> ) |
| <b>VM terminal</b>   | make -C ~/living_map/beacon_net/host clean<br>delete the built files                                                                                         |
| <b>VM or Windows</b> | python3 ~/living_map/beacon_net/python/tests/test_crosscheck.py<br>only the Python ⇄ C checks (Windows: run_tests.bat)                                       |

## H.3 Work with single frames (bptool)

The demo key "LivingMap-DEMO!!" in hex is 4c6976696e674d61702d44454d4f2121; the network id is 42 (0x2A). On Windows use bptool.exe in the windows folder (example\_frame.bat runs the first line). In ~/living\_map/beacon\_net/host:

|                    |                                                                                                                                                                                                                                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | ./bptool example<br>the reference VICTIM record from the brief, as JSON and as the 44-byte frame                                                                                                                                                                                                                                        |
| <b>VM terminal</b> | ./bptool fromjson 4c6976696e674d61702d44454d4f2121 42 '<br>{ "beacon_id": 7, "kind": "VICTIM", "gps": { "lat": 34.4312, "lon": 8.7845, "err_m": 2.1 }, "severity": 15, "next": { "id": 6, "dist_m": 8.5, "ttl_s": 7200 } }' 1758500000<br>any record as JSON → signed frame (the last number is the time stamp; leave it out for "now") |
| <b>VM terminal</b> | ./bptool decode 4c6976696e674d61702d44454d4f2121 42<br>212a05000f00050001000003e038f015579411060600ffbc02002300d83bb16ac021e01037f852f084ff7484<br>verify and decode a frame, e.g. one copied from a mission file (wrong key or one changed byte → rejected)                                                                            |
| <b>VM terminal</b> | ./bptool airtime 44 9 125000 5<br>time on air of a 44-byte frame at SF9, 125 kHz, CR 4/5: 288 ms                                                                                                                                                                                                                                        |
| <b>VM terminal</b> | ./bptool digest 4c6976696e674d61702d44454d4f2121 42 900 0 5:1 7:2<br>a signed pull digest from node 900 listing records 5 (seq 1) and 7 (seq 2)                                                                                                                                                                                         |
| <b>VM terminal</b> | ./bptool hmac 4c6976696e674d61702d44454d4f2121 00ff<br>raw HMAC-SHA256 of a hex message (test vectors)                                                                                                                                                                                                                                  |

# I. One gateway, no ONA: other ways into the dashboard

Without the ONA a single gateway (or a bridge playing one) posts straight to the dashboard. Quicker to set up, no vote and no robot positions. Start the dashboard first (part D), then one of these.

| source                                        | section | shows                                                          |
|-----------------------------------------------|---------|----------------------------------------------------------------|
| simulated beacon network (bpsim)              | I.1     | how the network behaves: gossip, lost beacons, aging, spoofing |
| the Writer in Gazebo (beacon_record_node)     | I.2     | the beacons the Writer drops, as it drops them                 |
| the Writer, original contract (ros_bridge.py) | I.3     | the same, older format (no LMB2 fields)                        |
| a real gateway board on USB                   | I.4     | real beacons over the air                                      |

## I.1 Simulated network to dashboard

|                    |                                                                                                                                                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Windows</b>     | <code>C:\tsyp\living_map\beacon_net\windows\simulate_to_dashboard.bat</code><br>simplest: bpsim at 20x real time straight into the Windows dashboard; beacons appear over a few minutes                                |
| <b>Windows</b>     | <code>py -3 ..\python\beaconnet\gateway_bridge.py --sim="--speed 20 --beacons 30 --rubble 2.5" --server http://localhost:3000</code><br>from cmd in beacon_net\windows, with any bpsim options                         |
| <b>VM terminal</b> | <code>cd ~/living_map/beacon_net/host &amp;&amp; ./bpsim --speed 20   python3 .<br/>./python/beaconnet/gateway_bridge.py --stdin --server http://10.0.2.2:3000</code><br>the simulator in the VM, dashboard on Windows |
| <b>VM terminal</b> | <code>cd ~/living_map/beacon_net/host &amp;&amp; ./bpsim --speed 20   python3 .<br/>./python/beaconnet/gateway_bridge.py --stdin --dry-run</code><br>no dashboard: print what would be posted                          |

## I.2 The Writer (Gazebo) to dashboard, LMB2 records

Run it in a new terminal while `run_explorer.sh` runs (without `ona`). It turns every dropped beacon into the same LMB2 record as the mission file and posts it, acting as a perfect radio network.

|                    |                                                                                                                                                  |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>python3 ~/living_map/beacon_net/ros/beacon_record_node.py --ros-args -p<br/>dashboard:=http://10.0.2.2:3000</code><br>dashboard on Windows |
| <b>VM terminal</b> | <code>python3 ~/living_map/beacon_net/ros/beacon_record_node.py --ros-args -p<br/>dashboard:=http://localhost:3000</code><br>dashboard in the VM |
| <b>VM terminal</b> | <code>ros2 topic echo /writer/beacon_record</code><br>the records it builds (JSON + frame hex)                                                   |

| parameter              | default          | meaning                                            |
|------------------------|------------------|----------------------------------------------------|
| dashboard              | (empty = off)    | dashboard address to post to                       |
| serial                 | (empty = off)    | USB port of a beacon to provision (K.4)            |
| anchor_lat, anchor_lon | 36.8065, 10.1815 | where the map origin is on Earth (the start point) |
| map_yaw_deg            | 0                | compass direction of the map +x axis (0 = east)    |
| key                    | demo             | mission key: demo or 32 hex characters             |
| net_id                 | 42               | network id (0x2A)                                  |

| parameter         | default   | meaning                                   |
|-------------------|-----------|-------------------------------------------|
| err0_m, err_per_m | 0.3, 0.02 | position error model                      |
| vision_confidence | 0.9       | confidence of camera detections           |
| gateway_id        | writer    | name of the gateway chip on the dashboard |

## I.3 The Writer to dashboard, original contract (ros\_bridge.py)

**VM terminal** `python3 ~/living_map/command_post/ros_bridge.py --ros-args -p server:=http://10.0.2.2:3000`  
 older bridge: posts each `/writer/beacon_dropped` in the original brief format (no LMB2 fields) with a gw1 heartbeat. Parameters: `anchor_lat`, `anchor_lon`, `map_yaw_deg`, `gateway_id`, `send_trail` (false = hazards only). Use I.2 or I.3, not both.

## I.4 A real gateway board to dashboard

**Windows** `C:\tsyp\living_map\beacon_net\windows\real_gateway_to_dashboard.bat`  
 asks for the COM port (Device Manager → Ports, e.g. COM5) and the mission key (Enter = demo), sends KEY and TIME to the board, then posts everything it hears

**Windows** `py -3 ..\python\beaconnet\gateway_bridge.py --serial COM5 --key demo --server http://localhost:3000`  
 the same by hand (from `beacon_net\windows`)

**VM terminal** `python3 ~/living_map/beacon_net/python/beaconnet/gateway_bridge.py --serial /dev/ttyUSB0 --key demo --server http://10.0.2.2:3000`  
 in the VM (needs Devices → USB passthrough of the board and the dialout group)

**VM terminal** `ls /dev/ttyUSB* /dev/ttyACM*`  
 which USB serial ports the VM sees

| gateway_bridge.py option                                         | meaning                                                                      |
|------------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>--serial PORT   --stdin   --sim "ARGS"   --exec CMD</code> | where the JSON lines come from (choose one)                                  |
| <code>--server URL</code>                                        | dashboard address (default <code>http://localhost:3000</code> )              |
| <code>--key HEX demo, --net N</code>                             | serial only: provision the gateway board with the mission key and network id |
| <code>--baud N</code>                                            | serial speed (115200)                                                        |
| <code>--gw NAME</code>                                           | gateway name on the dashboard (default from the board's heartbeat, else gw1) |
| <code>--dry-run · --quiet</code>                                 | print the payloads instead of posting · less output                          |

# J. The full demo, in order

## J.1 Make it fast enough for a demo

Everything the robots do is in simulated time. In the VM, Gazebo draws the 3D window, the camera and the LiDAR on the processor (no graphics card), so the simulation runs slower than real time: the Gazebo window shows it in its bottom-right corner (for example 20 % = five times slower). The Writer needs about 7.3 simulated minutes on the big map and the Executor about 3: at 20 % the big map takes about 35 minutes, at 60 % about 12. The robot logic is the same either way; the ONA follows the simulation speed by itself.

| what to do                                                                                                                                                                               | effect                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| run headless (no Gazebo window) and watch RViz                                                                                                                                           | the biggest gain: the 3D window is the heaviest part                |
| do not record the screen inside the VM; record on Windows (OBS Studio, Xbox Game Bar Win+Alt+R)                                                                                          | a recorder inside the VM takes a large share of the processor       |
| VM powered off → Settings → System: 4 or more processors, 8 GB memory; close other programs                                                                                              | the software renderer uses every core                               |
| live demo on the small map ( <code>retreat</code> , about 2 simulated minutes); show the big map as a video recorded beforehand, sped up                                                 | a demo that fits in a few minutes                                   |
| optional: VirtualBox Settings → Display → Enable 3D Acceleration, 128 MB, then add the option <code>gpu</code>                                                                           | can be much faster; if Gazebo fails to start, drop <code>gpu</code> |
| <b>VM terminal</b> <code>gz topic -e -t /world/contaminated_zone/stats -n 1   grep real_time_factor</code><br>how fast the simulation runs (1.0 = real time)                             |                                                                     |
| <b>VM terminal</b> <code>LIBGL_ALWAYS_SOFTWARE=1 gz sim -g</code><br>with a headless run: open the Gazebo window for a moment to show the 3D world; close it again to get the speed back |                                                                     |

## J.2 The order of the presentation

Dashboard first, a short warm-up on the ONA, then the Writer with its beacons appearing live, then the Executor briefed from the dashboard. Budget about 20 minutes on the big map in a VM, or 5–8 minutes with `retreat` everywhere.

|                    |                                                                                                                                                                                                                                                                                   |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Windows</b>     | <code>C:\ttyp\living_map\beacon_net\windows\run_dashboard.bat</code><br><b>1.</b> Start the Command Post. Keep the window open.                                                                                                                                                   |
| <b>Browser</b>     | <code>http://localhost:3000</code><br><b>2.</b> Open the dashboard.                                                                                                                                                                                                               |
| <b>Windows</b>     | <code>C:\ttyp\living_map\ona\windows\run_ona.bat --lte-outage 60-120</code><br><b>3.</b> Warm-up (optional, about 3 minutes), window A: the ONA with an LTE outage.                                                                                                               |
| <b>Windows</b>     | <code>C:\ttyp\living_map\ona\windows\simulate_faults.bat</code><br><b>4.</b> Window B: show the vote, the lying gateway, the forged victim, the crushed beacon, the SLAM drift and the satellite link. Then close windows A and B and restart the dashboard (step 1) to clear it. |
| <b>VM terminal</b> | <code>bash ~/writer_robot_ws/stop_demo.sh</code><br><b>5.</b> VM terminal 1: make sure nothing old is running.                                                                                                                                                                    |
| <b>VM terminal</b> | <code>~/writer_robot_ws/run_explorer.sh headless ona</code><br><b>6.</b> Terminal 1: the Writer (add <code>retreat</code> for the short version). Pins appear ✓3/3 as it drops beacons; the Writer moves on its three circles.                                                    |
| <b>VM terminal</b> | <code>tail -f /tmp/explorer.log</code><br><b>7.</b> Terminal 2: narrate: hazards, RETREAT, LOOK, EXPLORATION COMPLETE, DONE.                                                                                                                                                      |
| <b>Browser</b>     | (dashboard) click a hazard pin<br><b>8.</b> Show its route to the exit, its confidence and half-life, its vote.                                                                                                                                                                   |
| <b>VM terminal</b> | <code>Ctrl-C</code> in terminal 1 after DONE<br><b>9.</b> The mission file is saved ( <code>~/writer_robot_ws/missions/latest.json</code> ).                                                                                                                                      |
| <b>VM terminal</b> | <code>~/writer_robot_ws/run_executor.sh headless ona</code><br><b>10.</b> Terminal 1: the Executor waits at the exit ( <code>run_executor.sh demo headless ona</code> if the Writer run was cut short).                                                                           |

|                    |                                                                                                                                                                               |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Browser</b>     | (dashboard) Select on map → pick the targets → Dispatch mission<br><b>11.</b> "✓ acknowledged"; it treats them in your order; <b>Follow</b> shows the circles crossing on it. |
| <b>VM terminal</b> | <code>tail -f /tmp/executor.log</code><br><b>12.</b> Terminal 2: briefing, GOTO, approaching, TREATED, RETURN, MISSION COMPLETE.                                              |
| <b>Browser</b>     | (dashboard) Call the Executor back<br><b>13.</b> Optional: it turns back to the exit.                                                                                         |
| <b>VM terminal</b> | <code>bash ~/writer_robot_ws/stop_demo.sh</code><br><b>14.</b> End.                                                                                                           |

**TIP** Rehearse once with `retreat` to check timings. Short on time: skip the Writer and show `run_executor.sh demo headless ona`, with the big-map Writer as a video recorded beforehand. No VM at hand: `run_dashboard_demo.bat` alone tells the whole story on Windows, and `sim_mission_big_map.bat` shows both robots in 2D.

## K. Real hardware: beacons and gateways

### K.1 Arduino IDE 2 (Windows), once

- 1 File → Preferences → Additional boards manager URLs: add `https://espressif.github.io/arduino-esp32/package_esp32_index.json`, then Boards Manager → **esp32** by Espressif.
- 2 Library Manager: install **LoRa** by Sandeep Mistry.
- 3 Copy the folder `C:\tsyp\living_map\beacon_net\lib\BeaconProto` to `Documents\Arduino\libraries\BeaconProto`.

### K.2 Flash the boards

- 1 Open `beacon_net\firmware\gateway_node\gateway_node.ino`, pick your board (e.g. TTGO LoRa32) and COM port, **Upload**. Do it on three boards for the ONA (one is enough without it). The same sketch for all three: the ONA names them.
- 2 Open `beacon_net\firmware\beacon_node\beacon_node.ino` and upload it to every other board.
- 3 Alternative with PlatformIO: in each sketch folder run `pio run -t upload`.

### K.3 Talk to a board (Serial Monitor, 115200 baud)

|                       |                                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------------------|
| <b>Serial / board</b> | KEY demo<br>mission key (demo key; KEY <32 hex> for a real mission). Stored in flash.                    |
| <b>Serial / board</b> | TIME 1790000000<br>set the clock in unix seconds (beacons have no real-time clock; use the current time) |
| <b>Serial / board</b> | NET 42<br>network id (default 42)                                                                        |
| <b>Serial / board</b> | ID 7<br>node id (a beacon also takes it from PROV)                                                       |
| <b>Serial / board</b> | NAME gw1<br>gateway label shown on the dashboard                                                         |

|                       |                                                                                                                                                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Serial / board</b> | PROV {"beacon_id":7,"kind":"VICTIM","gps":{"lat":36.8066,"lon":10.1816,"err_m":2.1},"severity":15,"next":{"id":6,"dist_m":8.5}}<br>install this beacon's record: it starts announcing it within 0.8 s |
| <b>Serial / board</b> | SEED {"beacon_id":6,"kind":"WAYPOINT","gps":{"lat":36.8065,"lon":10.1815}}<br>give it a record it should already know                                                                                 |
| <b>Serial / board</b> | RETRACT<br>withdraw this beacon's event (RETRACTED flag, new seq)                                                                                                                                     |
| <b>Serial / board</b> | STATUS<br>one JSON line: id, role, key set, clock set, records known, duty used, frames sent and received (incl. bad MACs)                                                                            |
| <b>Serial / board</b> | DUMP<br>every record in the cache                                                                                                                                                                     |
| <b>Serial / board</b> | VERBOSE 1<br>print every frame received and sent (VERBOSE 0 to stop)                                                                                                                                  |
| <b>Serial / board</b> | RAW 1<br>gateway: raw frame lines for the ONA on (default; RAW 0 off)                                                                                                                                 |
| <b>Serial / board</b> | TX <88 hex characters><br>gateway: transmit one 44-byte frame (the ONA does it for briefings): OK TX type 0x24                                                                                        |
| <b>Serial / board</b> | REBOOT<br>restart (record and seq survive in flash)                                                                                                                                                   |
| <b>Serial / board</b> | FACTORY<br>erase key, id and record                                                                                                                                                                   |
| <b>Serial / board</b> | HELP<br>list the commands                                                                                                                                                                             |

A first real test: one gateway with I.4, one beacon on another port provisioned by hand with `KEY demo`, `TIME ...`, `PROV {...}`. Its pin appears on the dashboard. Unplug the beacon: after about 7-10 minutes it is marked "LOST?".

## K.4 Provision beacons from the Writer

|                    |                                                                                                                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>python3 ~/living_map/beacon_net/ros/beacon_record_node.py --ros-args -p serial:=/dev/ttyUSB1 -p key:=demo</pre><br>each time the Writer drops a beacon, the board on /dev/ttyUSB1 receives TIME, PROV with its record and SEED with the route so far |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## K.5 Three real gateways into the ONA

Place the three gateways around the building, as far apart as the site allows (not in a line), and survey each antenna's position. Copy `ona\ona_config.json` to `my_site.json` and replace each gateway's local position with its surveyed lat, lon, alt; set the mission key and the anchor (where the robots start, and the compass direction of their map +x).

|                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>"gateways": {   "gw1": {"lat": 36.806482, "lon": 10.181433, "alt": 12.0},   "gw2": {"lat": 36.806410, "lon": 10.181713, "alt": 12.2},   "gw3": {"lat": 36.806595, "lon": 10.181634, "alt": 12.0} }</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                    |                                                                                                                                                                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Windows</b>     | cd /d C:\tsyp\living_map\ona<br>in cmd: the ONA folder                                                                                                                                                                                    |
| <b>Windows</b>     | py -3 -m ona --serial gw1=COM3 --serial gw2=COM4 --serial gw3=COM5 --config my_site.json --command-post http://localhost:3000<br>the ONA sends each board KEY, NET, NAME, TIME itself, then votes on everything they hear                 |
| <b>VM terminal</b> | cd ~/living_map/ona && python3 -m ona --serial gw1=/dev/ttyUSB0 --serial gw2=/dev/ttyUSB1 --serial gw3=/dev/ttyUSB2 --config my_site.json --command-post http://10.0.2.2:3000<br>the same in the VM (USB passthrough of the three boards) |

**WATCH OUT** Locating the robots needs a time-of-flight ranging radio (SX1280 or UWB) on each gateway and robot. With plain LoRa boards the vote, the uplink and the briefings work; the robot positions come only from what the robots report.

## L. Command reference

### L.1 The commands you will use most

| what                         | command                                                                                                           | where   |
|------------------------------|-------------------------------------------------------------------------------------------------------------------|---------|
| install / update             | cd ~ && rm -rf ~/living_map && unzip -o ~/Downloads/living_map.zip then bash ~/living_map/writer_robot/install.sh | VM      |
| dashboard                    | run_dashboard.bat → http://localhost:3000                                                                         | Windows |
| Writer                       | ~/writer_robot_ws/run_explorer.sh [retreat small mine] [headless] [ona] [slow] [gpu] [ekf]                        | VM      |
| Executor                     | ~/writer_robot_ws/run_executor.sh [demo [retreat] mine FILE.json] [headless] [ona] [slow]                         | VM      |
| the Gafsa mine without ROS   | run_dashboard_mine_demo.bat · sim_mission_mine.bat                                                                | Windows |
| brief the Executor           | dashboard: Select on map → Dispatch mission                                                                       | browser |
| stop everything              | bash ~/writer_robot_ws/stop_demo.sh                                                                               | VM      |
| logs                         | tail -f /tmp/explorer.log · /tmp/executor.log · /tmp/ona.log                                                      | VM      |
| health of a run              | bash ~/writer_robot_ws/check_sim.sh · grep health /tmp/vision.log                                                 | VM      |
| ONA without ROS              | run_ona.bat + simulate_zone.bat / simulate_faults.bat                                                             | Windows |
| network sim → ONA            | bpsim_to_ona.bat                                                                                                  | Windows |
| whole mission without Gazebo | sim_mission_big_map.bat                                                                                           | Windows |
| all tests                    | run_all_tests.bat · ona\windows\run_tests.bat · make ... test / matrix                                            | either  |

### L.2 ROS 2 and Gazebo tools

|                    |                                                                                     |
|--------------------|-------------------------------------------------------------------------------------|
| <b>VM terminal</b> | ros2 topic list<br>every topic that exists right now                                |
| <b>VM terminal</b> | ros2 topic hz /scan<br>rate of a topic (LiDAR ≈ 8 Hz, camera ≈ 6 Hz, map every 2 s) |

|                    |                                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>ros2 topic echo /cmd_vel</code><br>what the robot brain commands                                                     |
| <b>VM terminal</b> | <code>ros2 topic info /camera/image_raw</code><br>who publishes and who listens                                            |
| <b>VM terminal</b> | <code>ros2 node list</code><br>every running node                                                                          |
| <b>VM terminal</b> | <code>ros2 lifecycle get /slam_toolbox</code><br>SLAM state: must be active                                                |
| <b>VM terminal</b> | <code>ros2 run tf2_ros tf2_echo map base_footprint</code><br>where the robot is on the map                                 |
| <b>VM terminal</b> | <code>ros2 run tf2_tools view_frames</code><br>draws the TF tree into frames_*.pdf in the current folder                   |
| <b>VM terminal</b> | <code>ros2 pkg prefix writer_robot</code><br>where the installed package is                                                |
| <b>VM terminal</b> | <code>gz topic -l</code><br>Gazebo's own topics (camera, LiDAR, pose)                                                      |
| <b>VM terminal</b> | <code>ps aux   grep -E "gz sim slam explorer executor ona"   grep -v grep</code><br>which simulation processes are running |

## M. Troubleshooting

### M.1 The robot does not drive, or the map in RViz turns or smears

In the simulation the odometry (odom→base\_footprint) is the robot's true motion from Gazebo, and SLAM builds the map straight from it. In RViz the robot must turn when it turns in Gazebo, and the walls must stay still. A map that is already corrupt does not heal: restart.

|                    |                                                                                                                                                                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <code>grep -c OdometryPublisher \$(ros2 pkg prefix writer_robot)/share/writer_robot/urdf/writer_robot.urdf.xacro</code><br>must print 1 (the installed robot publishes Gazebo's odometry); otherwise re-run <code>install.sh</code> |
| <b>VM terminal</b> | <code>bash ~/writer_robot_ws/check_sim.sh</code><br>while the robot runs: odom→base_footprint present and changing, /odom about 30 Hz, /scan about 8 Hz, /map every 2 s                                                             |
| <b>VM terminal</b> | <code>bash ~/writer_robot_ws/stop_demo.sh</code><br>then start again: a fresh SLAM map                                                                                                                                              |
| <b>VM terminal</b> | <code>tail -n 30 /tmp/slam.log</code><br>SLAM warnings (dropped messages, TF timeouts)                                                                                                                                              |
| <b>VM terminal</b> | <code>grep -E "recover blacklisted WARNING map check time so far" /tmp/explorer.log</code><br>where the robot got stuck and why, and where the time goes                                                                            |

Also make sure teleop is not running at the same time as the explorer.

## M.2 A hazard block is not detected

**VM terminal** `grep health /tmp/vision.log | tail -5`  
the last camera health lines

| the health line says                                                                 | meaning                                                          | what to do                                                                |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------|
| NO camera images                                                                     | the camera bridge is not running                                 | <code>tail /tmp/sim.log; ros2 topic hz /camera/image_raw</code>           |
| no depth images ... floor-contact range                                              | no depth: the range comes from where the block touches the floor | fine; detection still works                                               |
| colour frames > 0 but the colour never in "hazard frames" while the block is in view | HSV gate or blob size                                            | restart vision with <code>-p min_area_px:=400 (E.4)</code> ; drive closer |
| the colour is in "hazard frames" but no beacon                                       | it was farther than 4 m, or already marked within 1.5 m          | normal: the beacon comes when the robot is closer                         |

## M.3 The ONA and the dashboard

| you see                                    | check                                                                                                                                                                   |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dashboard stays empty                      | right address (10.0.2.2 from the VM, localhost on Windows); Node.js allowed on private networks; <code>curl -s http://10.0.2.2:3000/api/health</code> from the VM (D.4) |
| ONA panel "not heard", no pins             | <code>tail /tmp/ona.log</code> : a growing queue means the dashboard is not reachable (line above); <code>COMMAND_POST</code> right?                                    |
| gateways red "silent"                      | the ONA hears nothing: <code>cat /tmp/ona_link.log, ros2 topic echo /ona_link/status</code> ; only one ONA running?                                                     |
| "Address already in use" in /tmp/ona.log   | another ONA holds UDP 47100: <code>bash ~/writer_robot_ws/stop_demo.sh</code> (and close <code>run_ona.bat</code> windows), then start again                            |
| "the ONA is missing" when starting a robot | re-run <code>bash ~/living_map/writer_robot/install.sh</code> (it copies <code>ona/</code> into the package)                                                            |
| the Executor ignores the dispatch          | a mission dispatched before the ONA started is never replayed: dispatch again. <code>grep briefing /tmp/ona_link.log</code>                                             |
| it started before you dispatched           | <code>WAIT_BRIEFING=180 (F.3)</code> ; a briefing later still re-targets it                                                                                             |
| SLAM DRIFT on a robot                      | not a bug: its own map slipped (the RViz map smears, M.1). The dashboard follows the gateways; restart the run for a clean map.                                         |
| SATELLITE, gateways "sat"                  | LTE is down (or <code>--lte-outage</code> ): only the essentials go out, the rest waits in the queue                                                                    |
| dotted 1/3 pins                            | only one gateway reported that version: wait, or look for <code>GATEWAY_DISAGREES</code> in the feed                                                                    |
| pins from an old run                       | restart the dashboard (it keeps everything in memory)                                                                                                                   |

## M.4 Other problems

| symptom                                   | fix                                                                                                                   |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| ros2: command not found                   | <code>source /opt/ros/lyrical/setup.bash</code> (and check that A.1 added it to <code>~/.bashrc</code> )              |
| Package 'writer_robot' not found          | <code>source ~/writer_robot_ws/install/setup.bash</code> , or re-run B.1, then open a new terminal                    |
| install ends without "1 package finished" | scroll up to the first red error; re-run the install line; send the error text                                        |
| the script waits forever for /scan        | <code>tail /tmp/sim.log; bash ~/writer_robot_ws/stop_demo.sh</code> and start again (an old Gazebo was still running) |

| symptom                                                                   | fix                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| waiting for map->odom never ends                                          | <code>ros2 lifecycle get /slam_toolbox</code> ; if not active: <code>ros2 lifecycle set /slam_toolbox activate</code>                                                                                                                                                             |
| robot does not move at all                                                | <code>tail /tmp/explorer.log</code> : it waits for map + pose + scan; run <code>check_sim.sh</code>                                                                                                                                                                               |
| script says no odom -> base_footprint TF                                  | <code>grep -i odom /tmp/sim.log</code> ; the URDF must load the OdometryPublisher (re-run <code>install.sh</code> ); last resort: the option <code>ekf</code>                                                                                                                     |
| EXPLORATION COMPLETE right after the start                                | <code>grep "map check" /tmp/explorer.log</code> and send that line                                                                                                                                                                                                                |
| a beacon is in Gazebo but not in RViz                                     | check the "Beacons" display is on; <code>tail /tmp/beacon.log</code> , <code>cat /tmp/beacon_spawn.log</code>                                                                                                                                                                     |
| WARNING: a full turn was commanded but the robot heading ... changed only | the odometry does not see the turn: <code>ros2 topic hz /odom</code> ; restart with <code>stop_demo.sh</code>                                                                                                                                                                     |
| Gazebo very slow, real-time factor low                                    | <code>headless</code> ; more CPU for the VM; close RViz panels you do not need (J.1)                                                                                                                                                                                              |
| camera looks, recoveries or driving take long                             | <code>grep -E "time so far recover" /tmp/explorer.log</code> . "recover: blocked" = it pushed against something the LiDAR does not see; "no progress" = 8 s without getting closer. If it bumps into things, add <code>slow</code> .                                              |
| !! no mission file (Executor)                                             | run the Writer first (it writes <code>latest.json</code> ), or <code>run_executor.sh demo</code>                                                                                                                                                                                  |
| Executor stays in WAIT                                                    | <code>cat /tmp/exec_radio.log</code> : frames verified? A key mismatch rejects every frame; the EXIT beacon must be in the file. With <code>ona</code> it waits up to <code>WAIT_BRIEFING</code> seconds for a briefing.                                                          |
| Executor skips a target (RViz: "skipped")                                 | <code>grep -E "skipped approach failed unreachable attempt" /tmp/executor.log</code> says why: no standoff point reachable, stuck, no view of it, not connected to the exit, or faded. It tries up to 4 points, treats from up to 3 m if in sight, and retries before going home. |
| <code>bpsim.exe</code> blocked                                            | SmartScreen: More info → Run anyway                                                                                                                                                                                                                                               |
| COM port busy                                                             | close the Arduino Serial Monitor before the bridge or the ONA opens the port                                                                                                                                                                                                      |
| a pasted command does strange things                                      | paste one line at a time, from <code>COMMANDS.txt</code>                                                                                                                                                                                                                          |

## M.5 Logs and files

| file (VM)                                                                       | written by                                                                                                  |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>/tmp/sim.log</code> , <code>/tmp/exec_sim.log</code>                      | Gazebo and the bridges (Writer run / Executor run)                                                          |
| <code>/tmp/slam.log</code> , <code>/tmp/exec_slam.log</code>                    | <code>slam_toolbox</code>                                                                                   |
| <code>/tmp/explorer.log</code>                                                  | the Writer brain                                                                                            |
| <code>/tmp/vision.log</code> , <code>/tmp/exec_vision.log</code>                | camera node, with [health] lines every 10 s                                                                 |
| <code>/tmp/beacon.log</code> , <code>/tmp/beacon_spawn.log</code>               | beacon drops and links; every Gazebo beacon spawn (errors show here)                                        |
| <code>/tmp/mission.log</code>                                                   | records written to the mission file                                                                         |
| <code>/tmp/frame.log</code>                                                     | lat/lon translator                                                                                          |
| <code>/tmp/executor.log</code>                                                  | the Executor brain                                                                                          |
| <code>/tmp/exec_radio.log</code>                                                | the simulated LoRa receiver                                                                                 |
| <code>/tmp/mine_sensors.log</code> ,<br><code>/tmp/exec_mine_sensors.log</code> | the mine sensor suite (option <code>mine</code> ): air readings, sightings and the victim search every 10 s |
| <code>/tmp/ona.log</code>                                                       | the ONA started by the <code>ona</code> option                                                              |
| <code>/tmp/ona_link.log</code>                                                  | the robot's radio to the three gateways                                                                     |
| <code>/tmp/ona_queue.sqlite3</code>                                             | the ONA's queue and audit log of the last run (G.4)                                                         |

| file (VM)                                              | written by                                                                                            |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| /tmp/rviz.log, /tmp/exec_rviz.log                      | RViz                                                                                                  |
| ~/writer_robot_ws/missions/latest.json                 | the last Writer mission (beacons, records, frames)                                                    |
| ~/writer_robot_ws/missions/mission_<date>.json         | a copy of every Writer run                                                                            |
| ~/writer_robot_backups/                                | previous packages and scripts, never deleted                                                          |
| /tmp/executor_world.world                              | the Executor's world with the beacons in place                                                        |
| config/ona_gazebo_big.json,<br>ona_gazebo_retreat.json | gateway positions for the two Gazebo maps (in the package; the ONA and the robots read the same file) |

## N. The Gafsa mine (tunnels)

The same system, underground: a simulated room-and-pillar phosphate panel of the kind CPG worked at Métlaoui, Redeyef and Moularès. A mine sensor suite (multi-gas, radon and gamma, thermal camera, roof scanner, XRF wall probe) replaces the colour-block vision; beacons also mark phosphate layers with their grade, mineral finds and **searched areas** ("searched, N victims"); the Executor reaches trapped miners first; the dashboard draws the mine plan. *3\_Gafsa\_Mine\_Case.pdf* explains the dangers, the sensors and the robot specification.

### N.1 Without ROS

|                    |                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Windows</b>     | C:\tsyp\living_map\beacon_net\windows\run_dashboard_mine_demo.bat<br>the dashboard + the mine mission replayed: the plan, beacons appearing, areas turning green; then the Executor waits at the portal: <b>Select on map → Dispatch mission</b> within a minute (e.g. the rich phosphate layer and the miner), else it goes for the miner first and every danger |
| <b>Browser</b>     | http://localhost:3000<br>the dashboard (Gafsa mine panel on the right, <b>Resources</b> and <b>Search</b> buttons on the map)                                                                                                                                                                                                                                     |
| <b>Windows</b>     | C:\tsyp\living_map\writer_robot\windows\sim_mission_mine.bat<br>the whole mine mission in the 2D simulator, with its checks and a picture                                                                                                                                                                                                                         |
| <b>Windows</b>     | C:\tsyp\living_map\writer_robot\windows\sim_mine.bat<br>only the Writer in the mine                                                                                                                                                                                                                                                                               |
| <b>Windows</b>     | C:\tsyp\living_map\ona\windows\run_ona_mine.bat<br>the real ONA for the mine (window 1, dashboard started first)                                                                                                                                                                                                                                                  |
| <b>Windows</b>     | C:\tsyp\living_map\ona\windows\simulate_mine.bat<br>window 2: the recorded mine mission as the three gateways hear it                                                                                                                                                                                                                                             |
| <b>VM terminal</b> | cd ~/living_map/command_post && npm run demo:mine<br>the dashboard mine demo in the VM                                                                                                                                                                                                                                                                            |
| <b>VM terminal</b> | cd ~/writer_robot_ws/src/writer_robot && python3 tools/sim_mission.py --world worlds/gafsa_mine.world --png /tmp/mine.png<br>the whole mine mission in 2D: victims first, every area searched, every resource marked                                                                                                                                              |
| <b>VM terminal</b> | cd ~/writer_robot_ws/src/writer_robot && python3 tools/sim_mission.py --world worlds/gafsa_mine.world --mission missions/demo_mine.json --wait-briefing 30 --briefing 5:51,53<br>a briefing: sample the rich phosphate layer #51, then the miner #53                                                                                                              |
| <b>VM terminal</b> | cd ~/living_map/ona && python3 -m ona --config ona_config_gazebo_mine.json --udp 0.0.0.0:47100 --command-post http://10.0.2.2:3000<br>the mine ONA in the VM (terminal 1)                                                                                                                                                                                         |

|                    |                                                                                                                                                                                                                    |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>cd ~/living_map/ona &amp;&amp; ./simulate_zone.sh --config ona_config_gazebo_mine.json --mission ../writer_robot/missions/demo_mine.json</pre> <p>terminal 2: the mine mission through the three gateways</p> |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## N.2 With the Gazebo robots

|                    |                                                                                                                                                                                                                               |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_explorer.sh mine headless ona</pre> <p>the Writer in the Gafsa mine, with its mine sensor head: dangers, resources and searched areas get beacons; the dashboard (Windows) draws the mine plan</p> |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                    |                                                                                                                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>tail -f /tmp/mine_sensors.log</pre> <p>every 10 s: CO, NO<sub>2</sub>, H<sub>2</sub>S, CH<sub>4</sub>, O<sub>2</sub>, radon, gamma, the sightings and the search coverage of each area</p> |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>ros2 topic echo /writer/air</pre> <p>the air readings, once a second</p> |
|--------------------|-------------------------------------------------------------------------------|

|                    |                                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>grep -E "MINE EVENT RESOURCE VICTIM SEARCH" /tmp/mine_sensors.log</pre> <p>every beacon event of the mine sensors</p> |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|

|                    |                                                                                                                                                                                                                |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_executor.sh headless ona</pre> <p>after the Writer: the Executor in the same mine (the mission file names the world): miners first, roofs propped, bad air and radon ventilated</p> |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                    |                                                                                                                         |
|--------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>~/writer_robot_ws/run_executor.sh mine headless ona</pre> <p>the ready-made mine mission, no Writer run needed</p> |
|--------------------|-------------------------------------------------------------------------------------------------------------------------|

|                |                                                                                                                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Browser</b> | <p>(dashboard) click a phosphate pin → Add to mission → Dispatch mission</p> <p>the Executor goes to take a sample ("taking a phosphate sample"); resources are targets only when briefed</p> |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## N.3 Change the mine

|                    |                                                                                                                                                                                                                                                                                                                             |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM terminal</b> | <pre>cd ~/living_map/writer_robot &amp;&amp; python3 tools/make_mine.py &amp;&amp; bash install.sh</pre> <p>after editing the layout, dangers, resources, search areas or gateways in <code>tools/make_mine.py</code>: rewrites the world, the site file, the dashboard plan and the mine ONA configs, then re-installs</p> |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## N.4 Troubleshooting in the mine

| you see                                                | check                                                                                                                                                                                                   |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| the dashboard shows streets, no mine plan              | the ONA must run with the mine config (the <code>mine</code> option does it; on Windows <code>run_ona_mine.bat</code> ); without the ONA: <code>SITE=gafsa_mine npm start</code> ; then reload the page |
| <code>/tmp/mine_sensors.log</code> says "no /odom yet" | the simulation is not running: <code>tail /tmp/sim.log</code>                                                                                                                                           |
| no SEARCHED beacon for an area                         | an area counts as searched at 85 % seen by the thermal camera: the Writer did not look at all of it (a danger kept it out?); the Gafsa mine panel shows it grey                                         |
| a robot "own report only"                              | normal underground: no gateway sees it straight down a gallery                                                                                                                                          |
| a gold or gemstone pin                                 | demonstration targets: none is documented in the Gafsa basin                                                                                                                                            |